

COMP3026E Distributed Systems

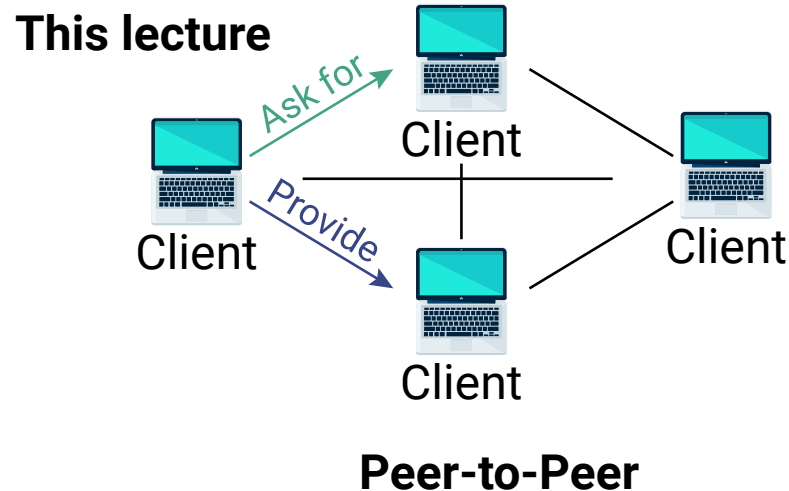
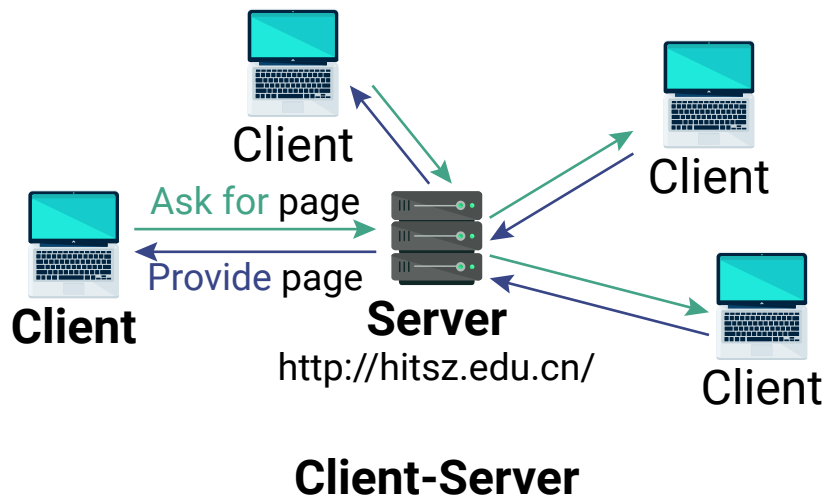
6. P2P Systems and Chord

James Jianqiao Yu

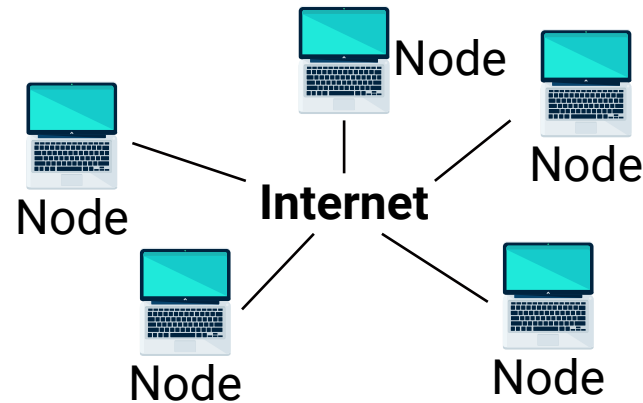
Harbin Institute of Technology, Shenzhen

1. Peer-to-peer (P2P) systems
2. Distributed hash tables (DHTs)
3. Chord: a DHT protocol
4. Concluding thoughts on DHTs and P2P systems

Distributed Application Architecture



What is a Peer-to-Peer (P2P) System?

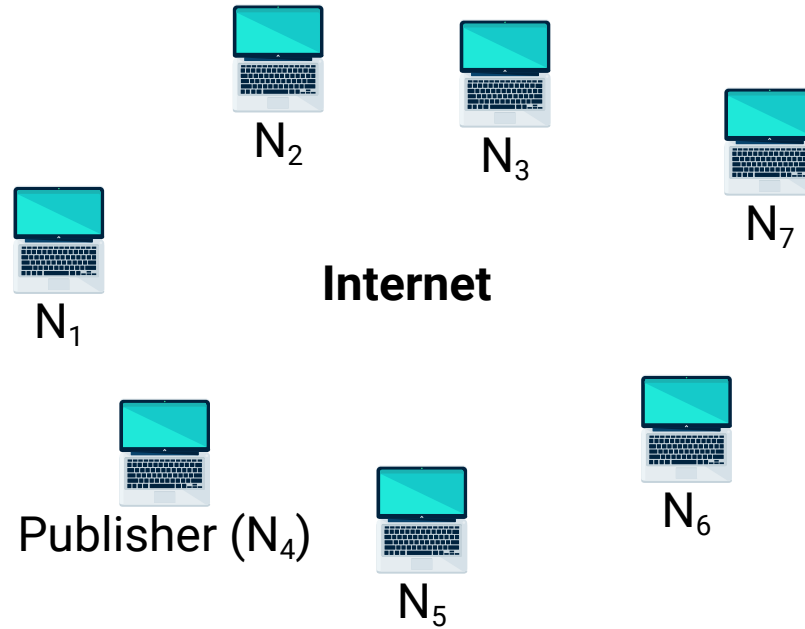


- A **distributed** system architecture:
 - **No centralized control**
 - Nodes are **roughly symmetric** in function
- Large number of **unreliable** nodes

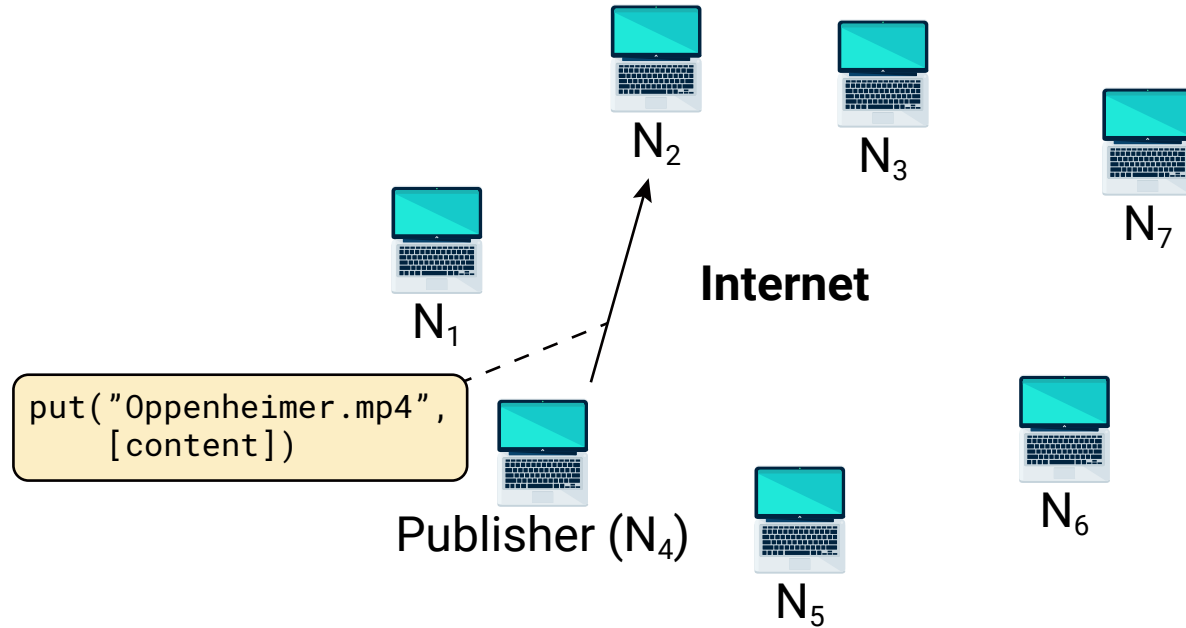
- Successful adoption in some niche areas
 1. Client-to-client (legal, illegal) **file sharing**
 - Napster (1990s), Gnutella, BitTorrent, etc.
 2. **Digital currency**: no natural single owner (Bitcoin)
 3. **Voice/video telephony**: user to user anyway
 - Issues: Privacy and control



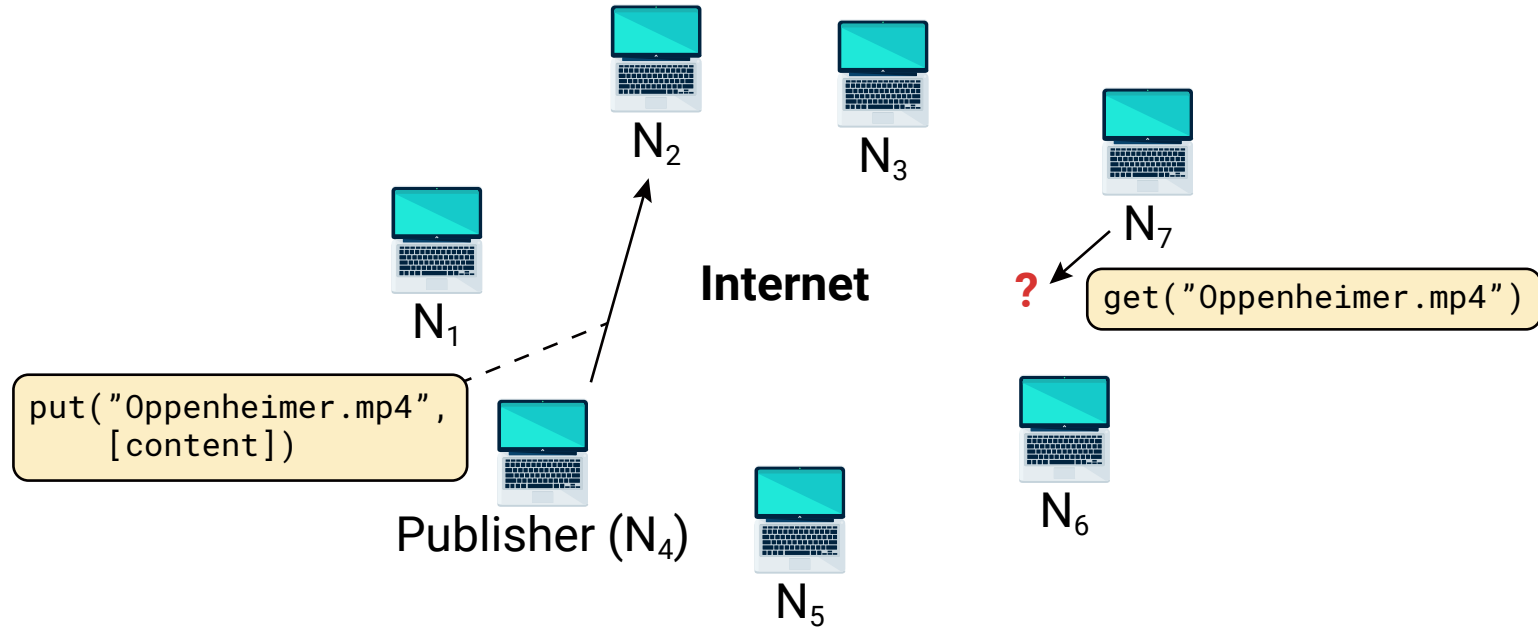
The Lookup Problem: Locate the Data



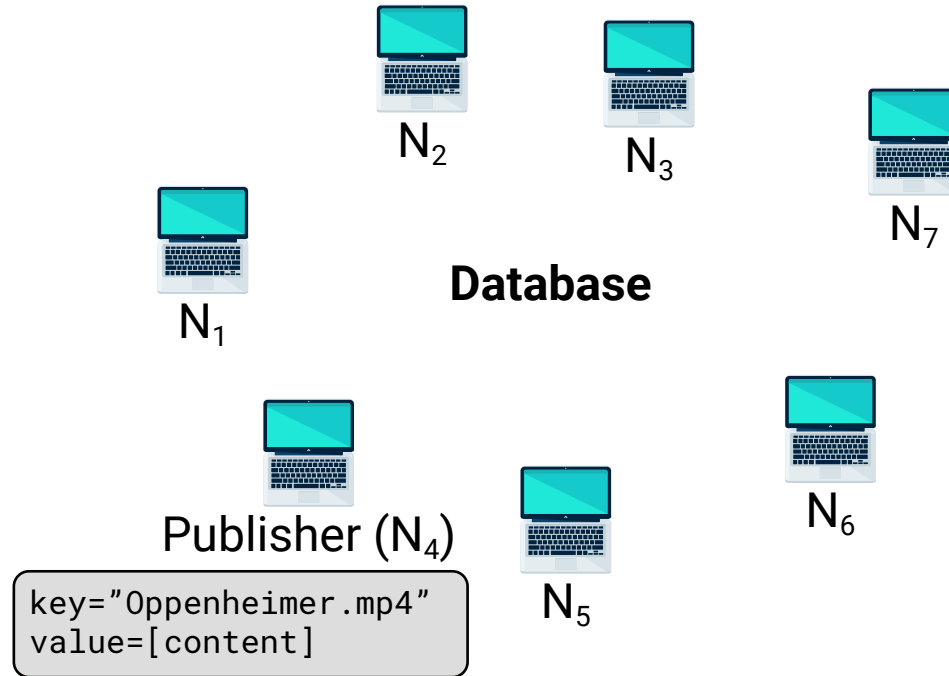
The Lookup Problem: Locate the Data



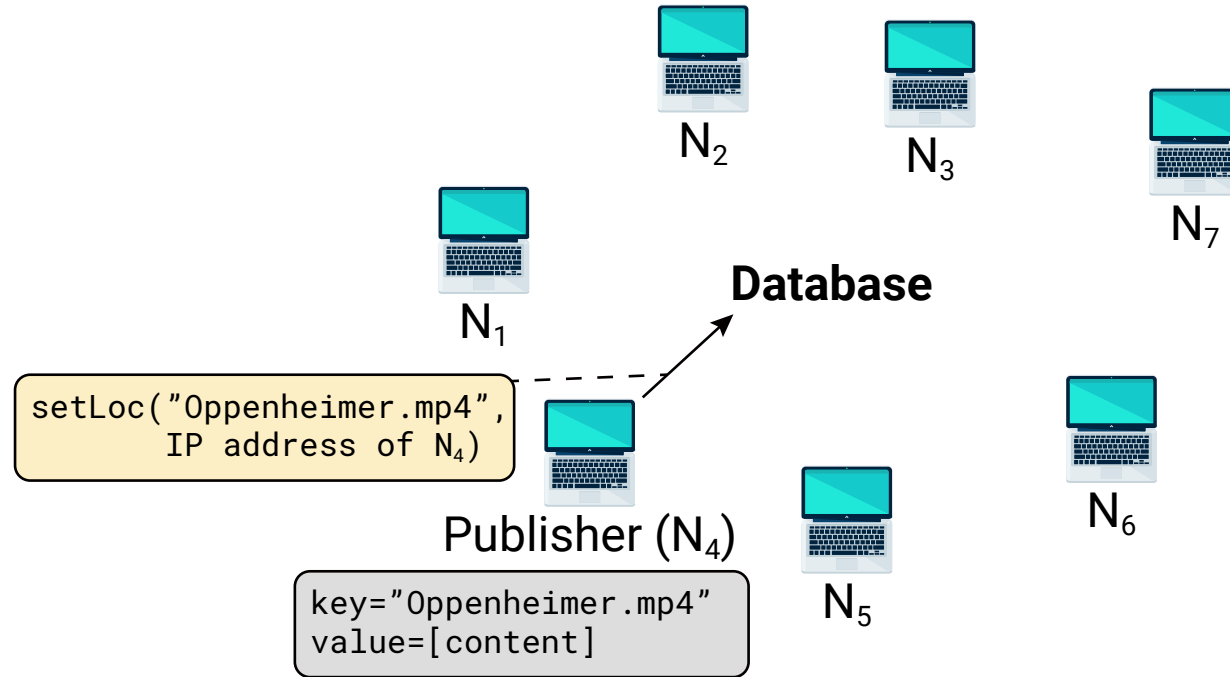
The Lookup Problem: Locate the Data



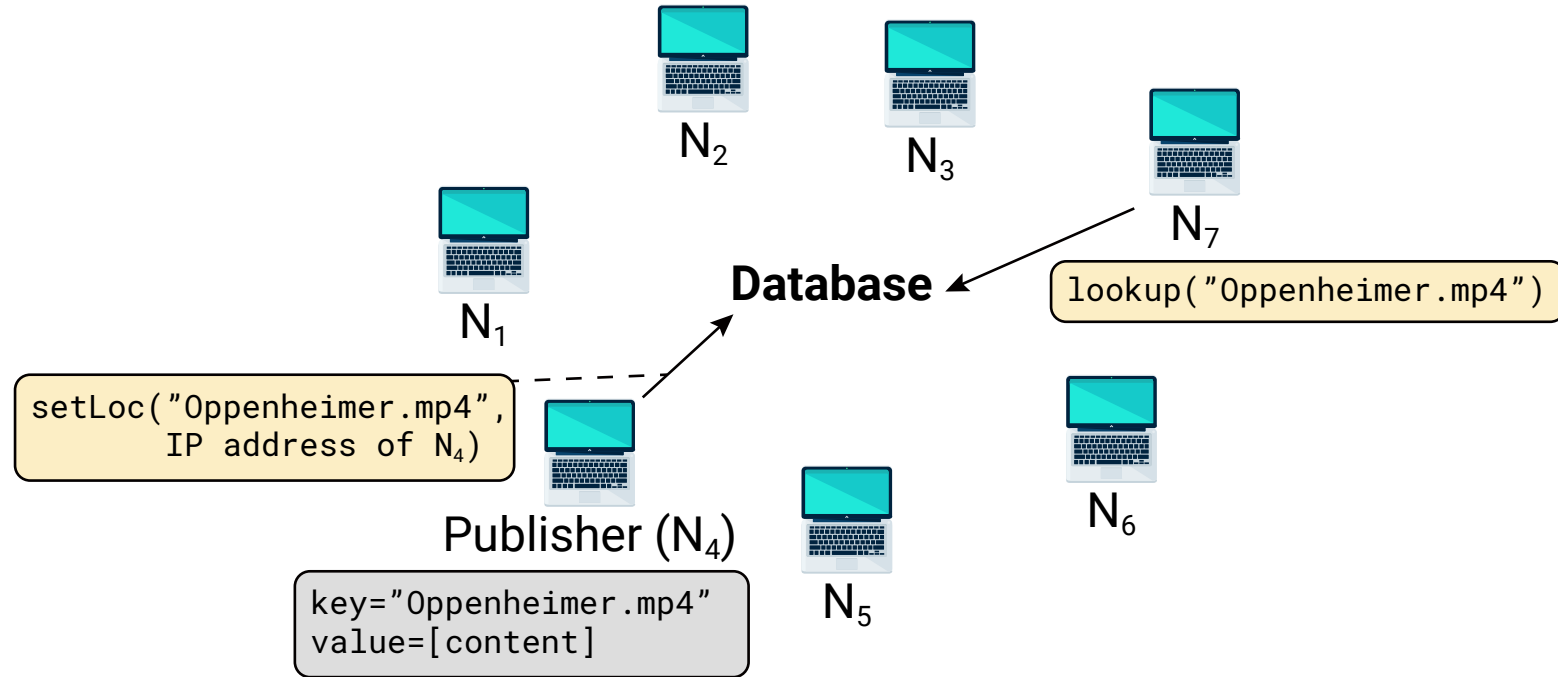
Centralized Lookup (Napster)



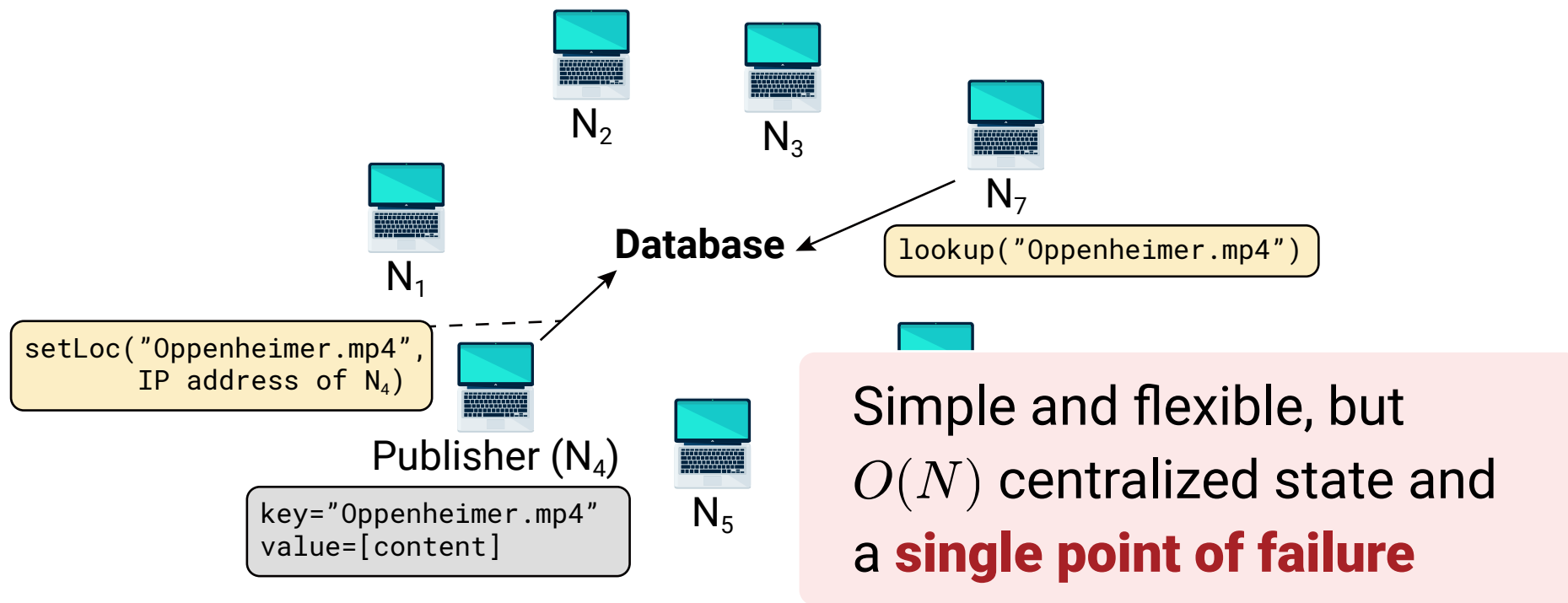
Centralized Lookup (Napster)



Centralized Lookup (Napster)

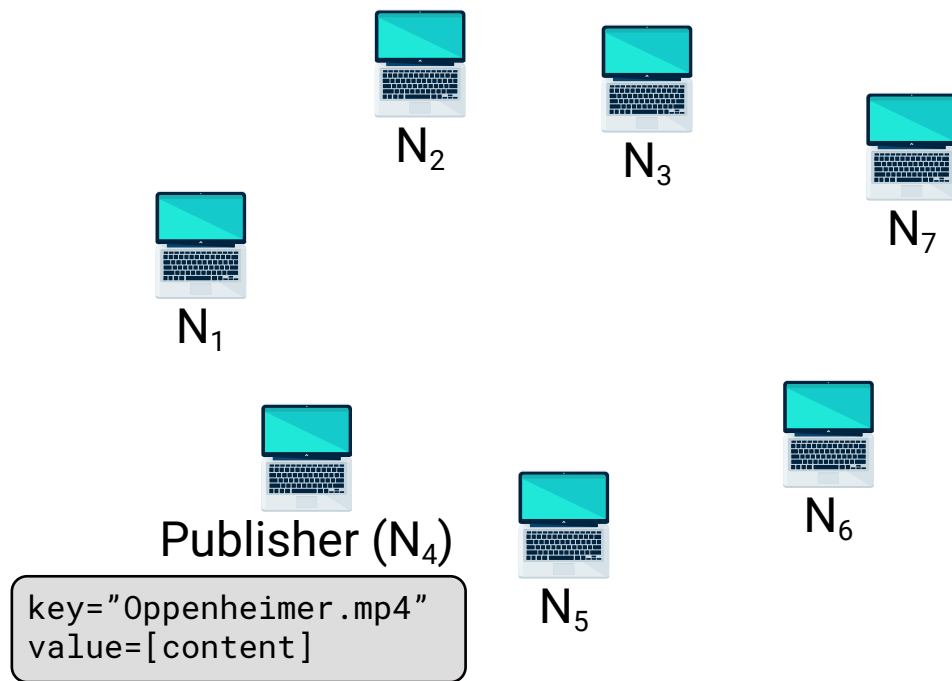


Centralized Lookup (Napster)

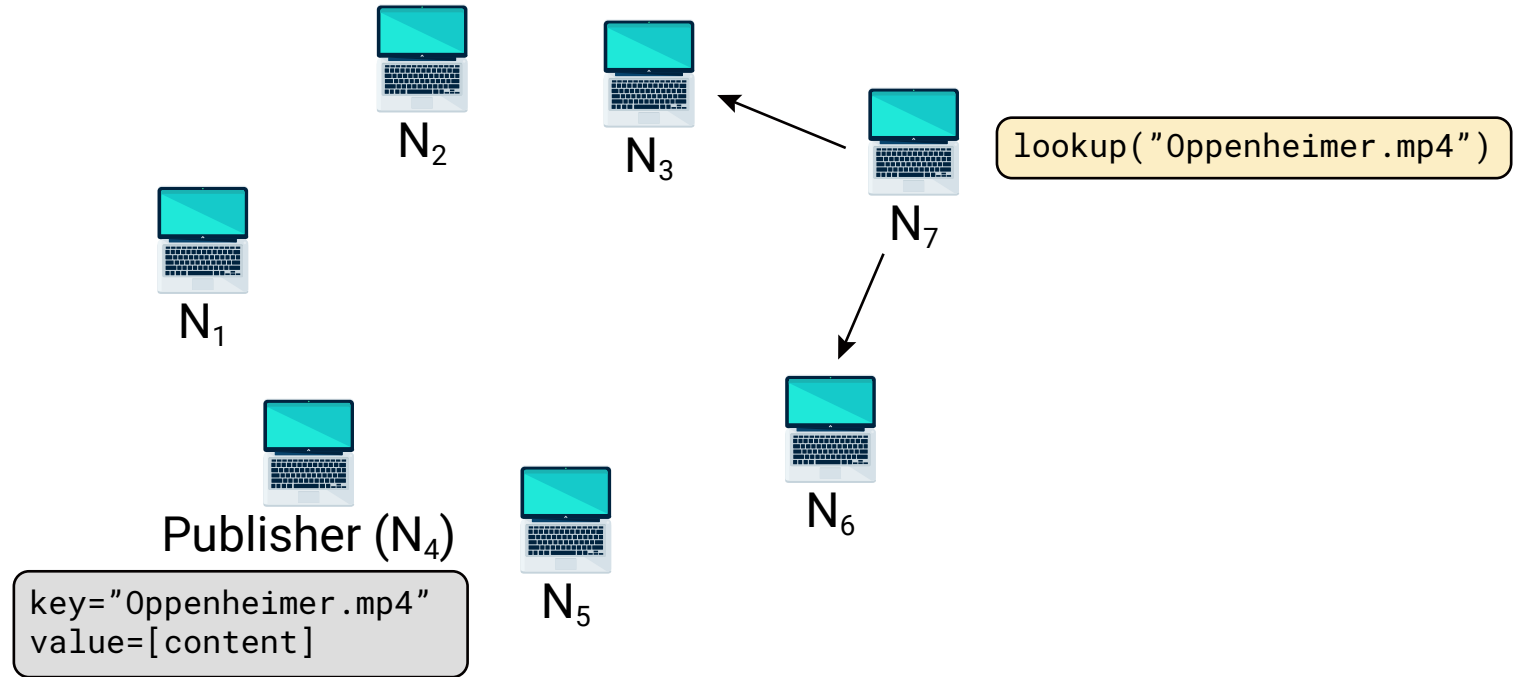




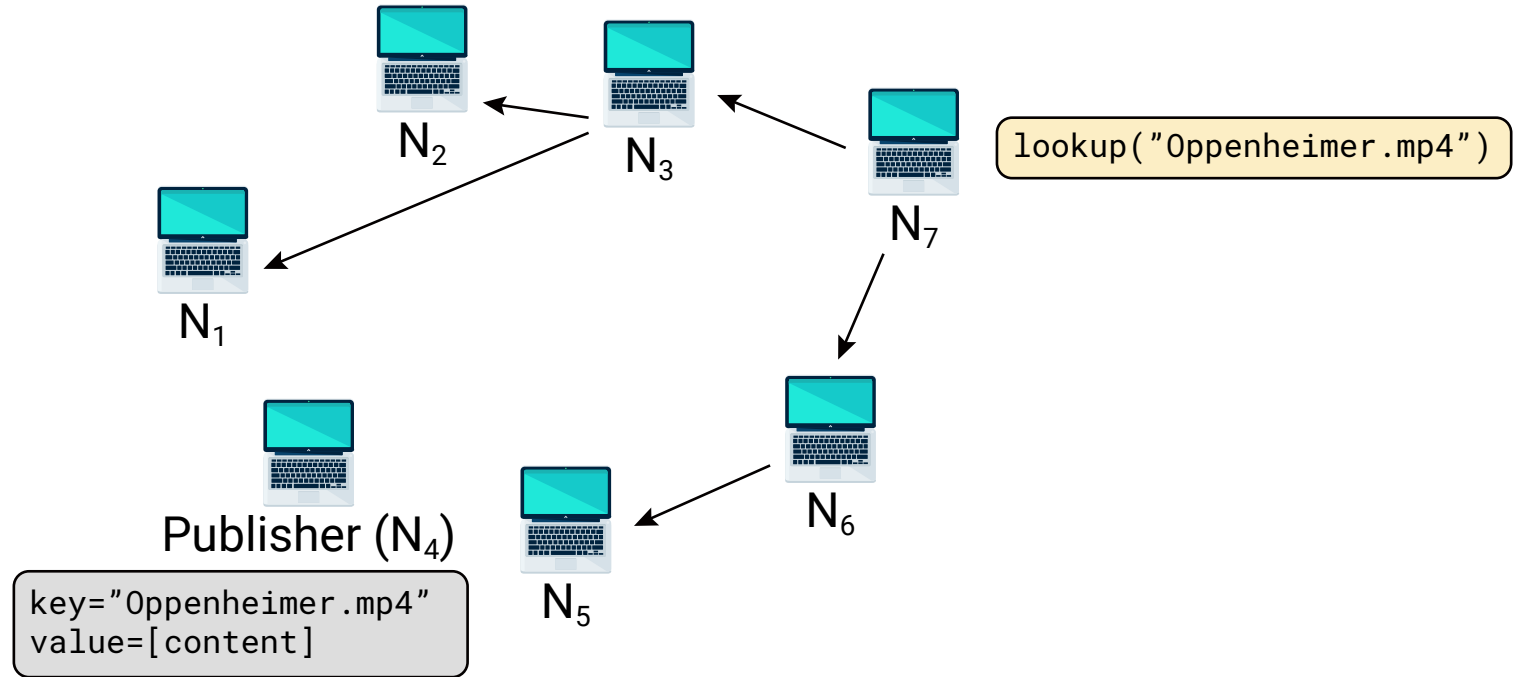
Flooded Queries (Original Gnutella)



Flooded Queries (Original Gnutella)

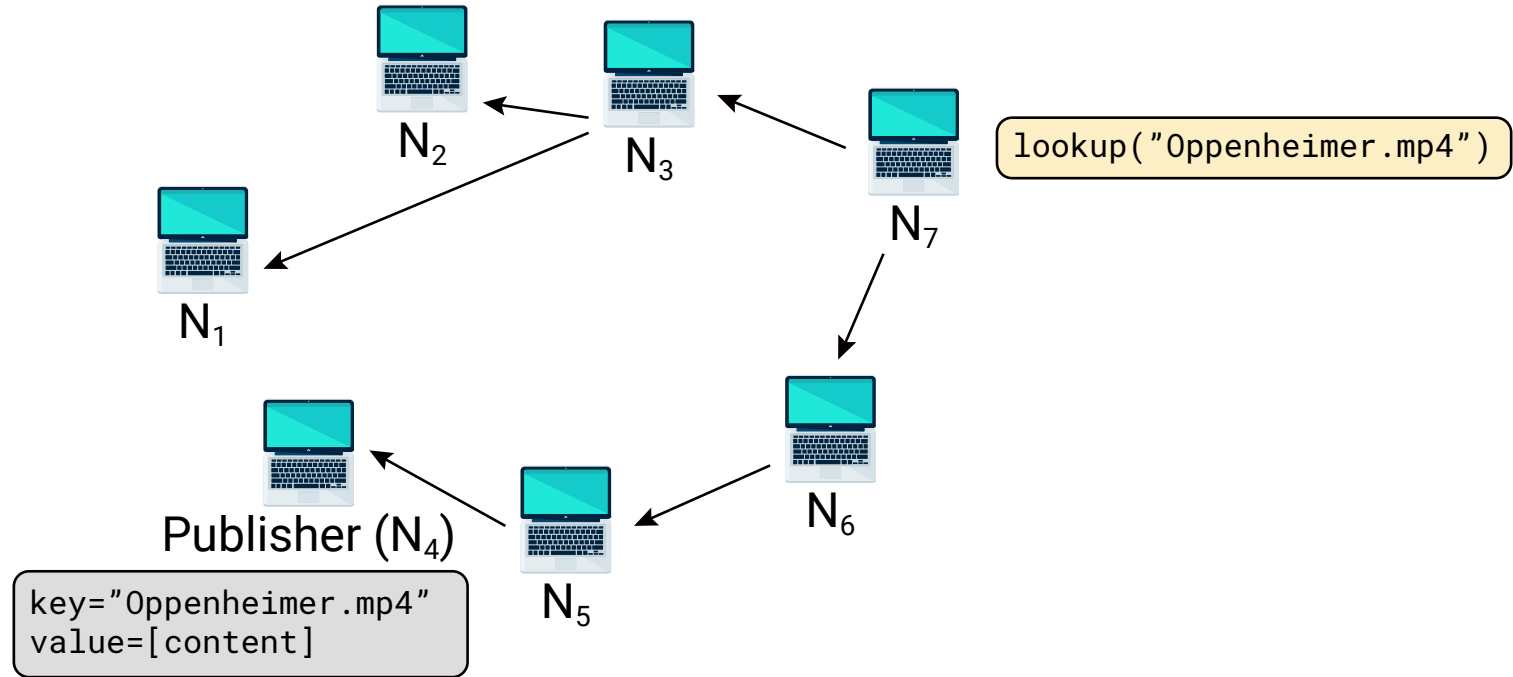


Flooded Queries (Original Gnutella)



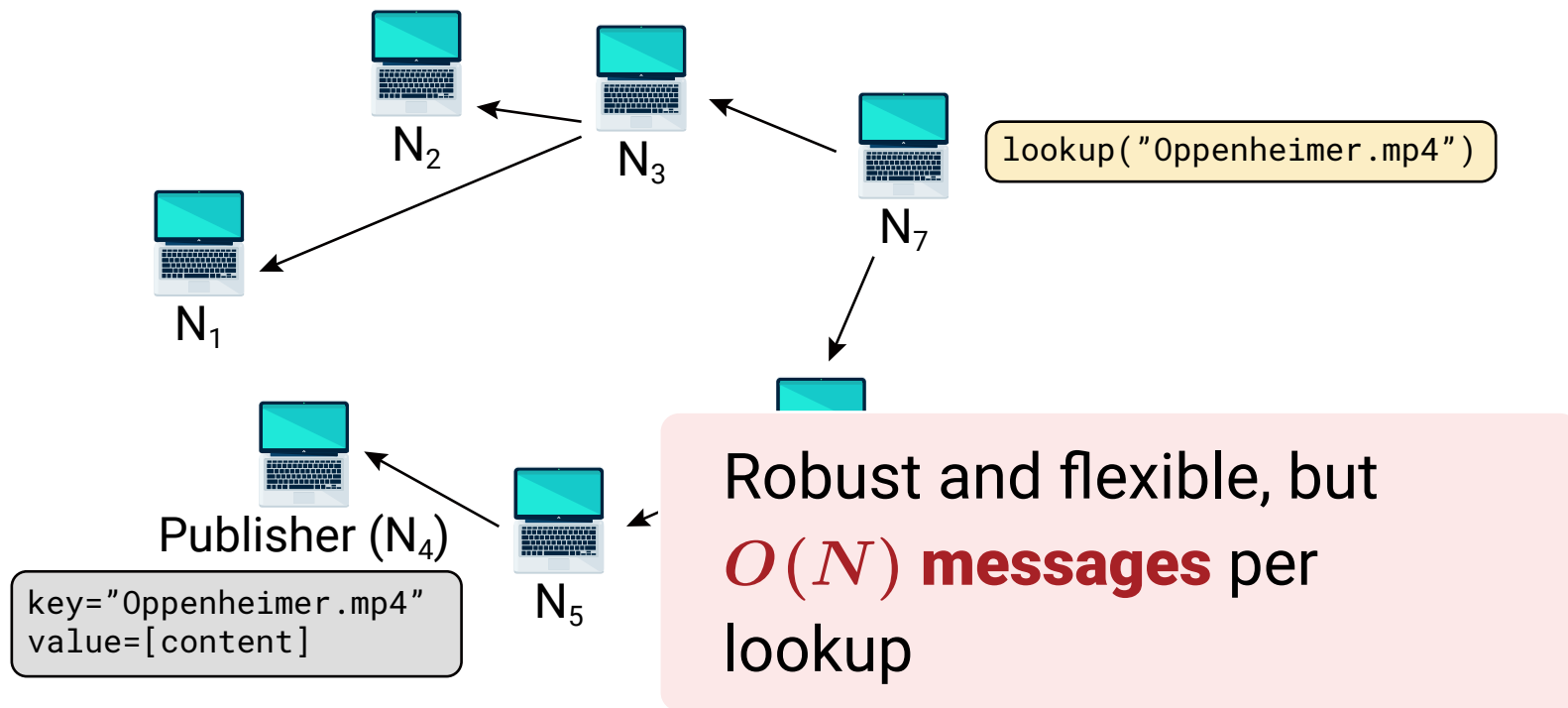


Flooded Queries (Original Gnutella)



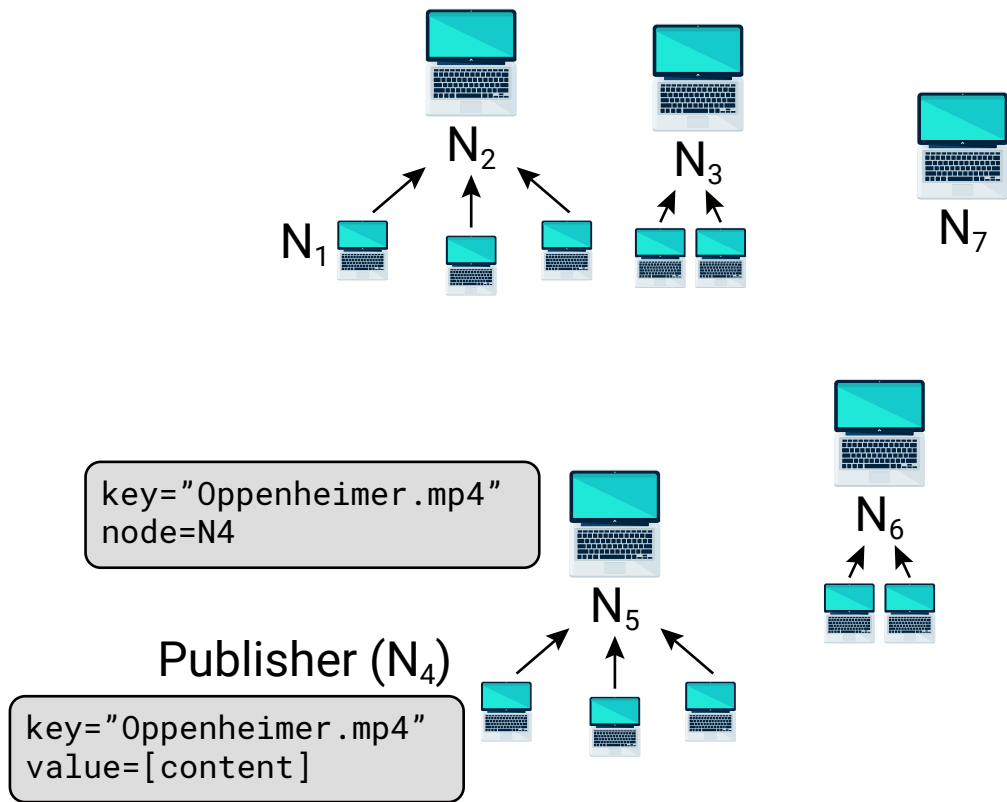


Flooded Queries (Original Gnutella)



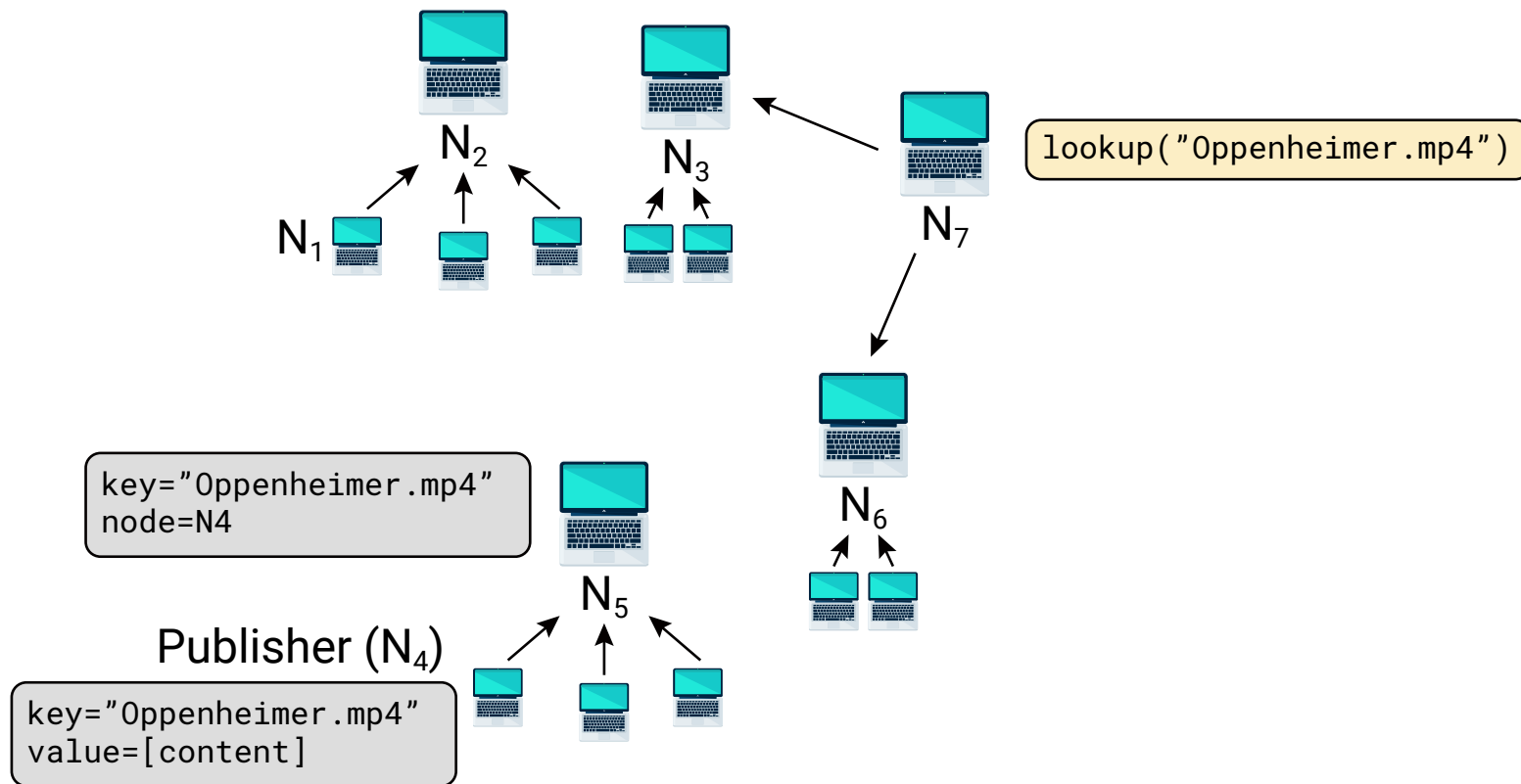


Flooded Queries Pt. 2 (Gnutella w/ SuperPeers)



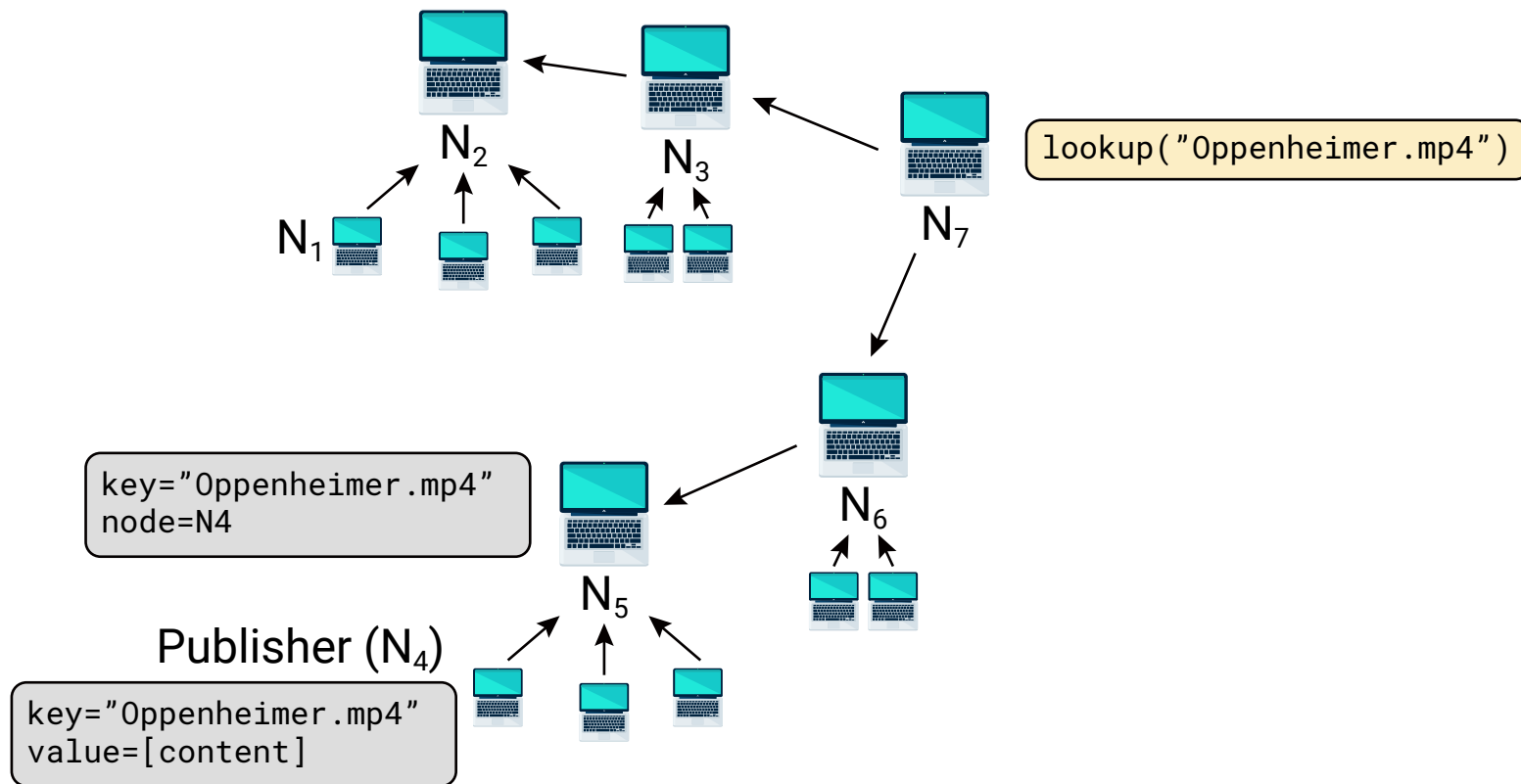


Flooded Queries Pt. 2 (Gnutella w/ SuperPeers)



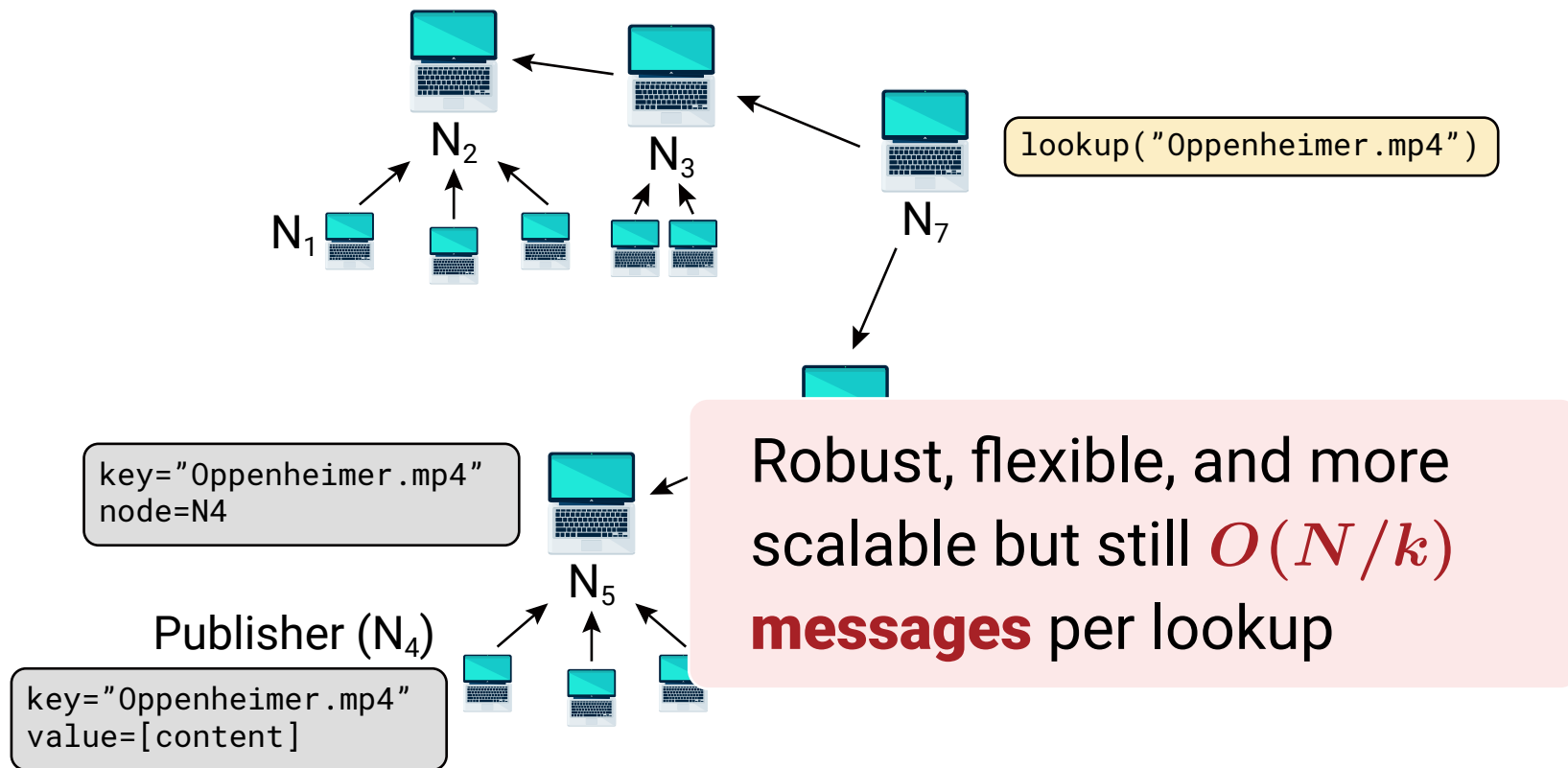


Flooded Queries Pt. 2 (Gnutella w/ SuperPeers)



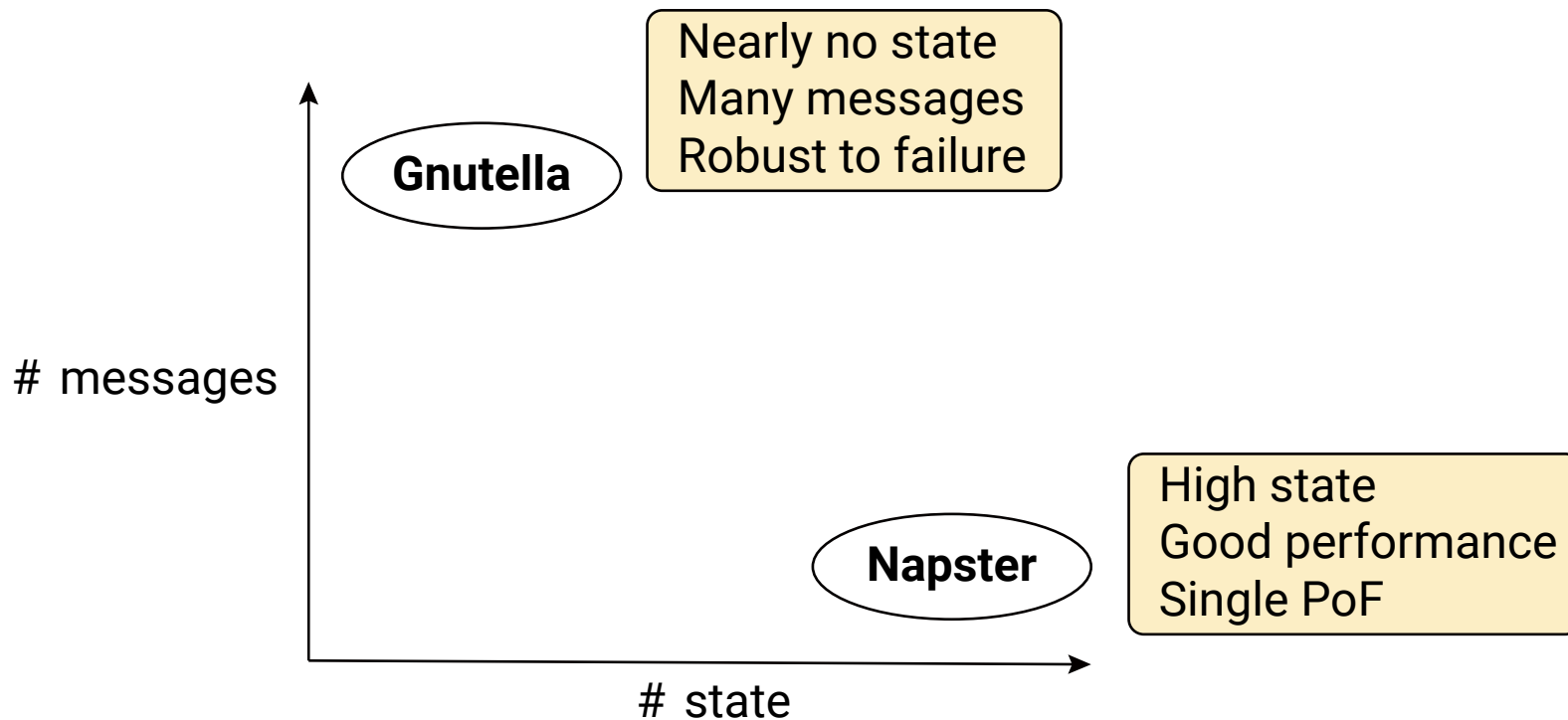


Flooded Queries Pt. 2 (Gnutella w/ SuperPeers)



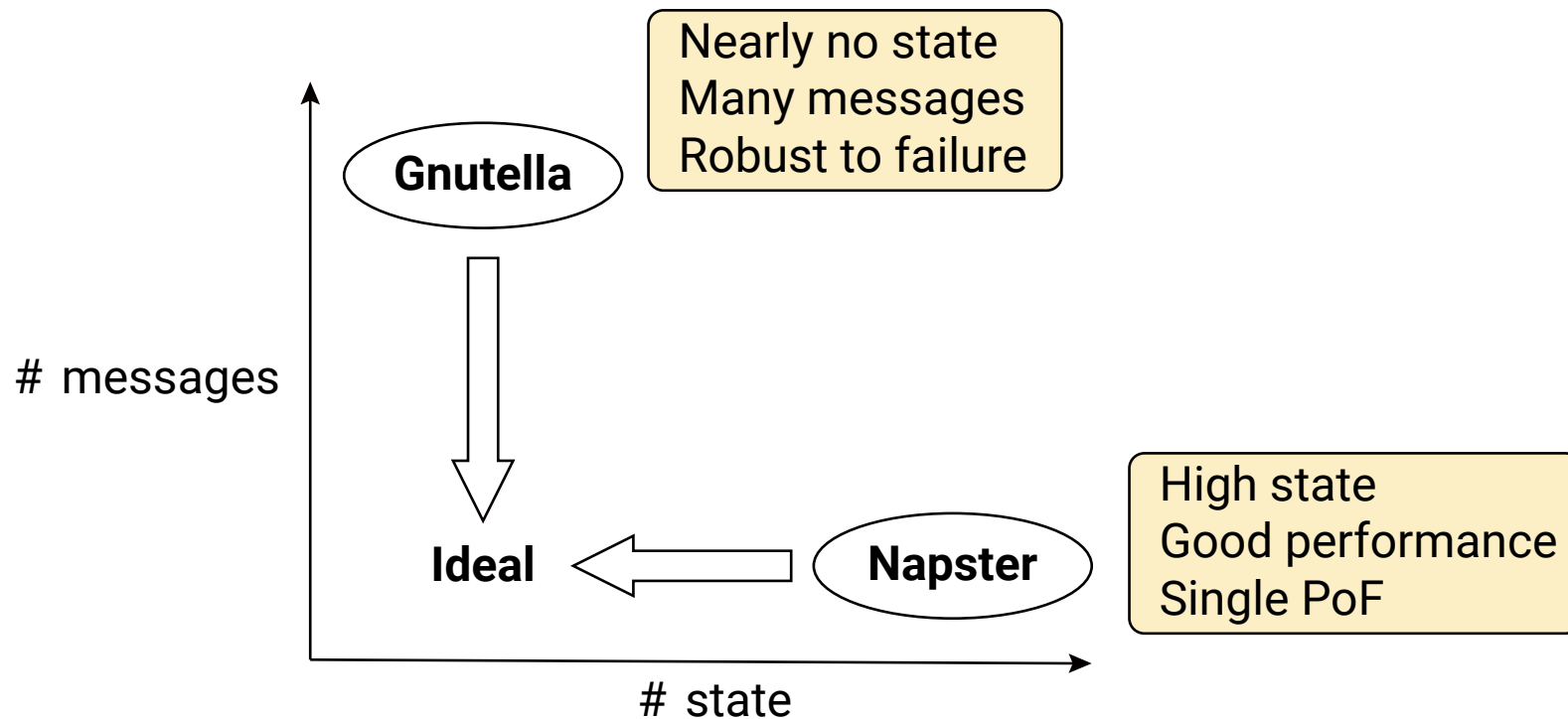


Tradeoffs in Distributed Systems



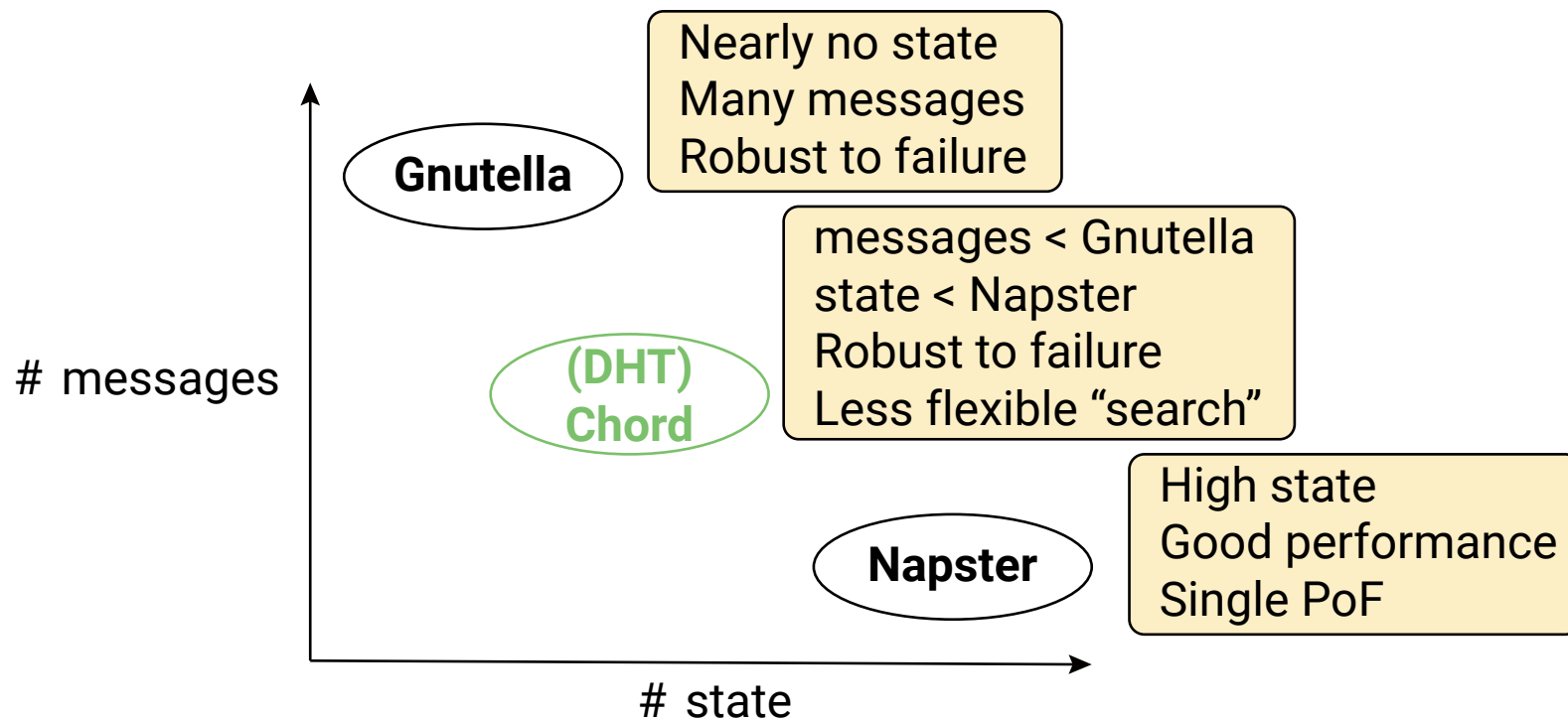


Tradeoffs in Distributed Systems





Tradeoffs in Distributed Systems



What is a DHT (and Why)?

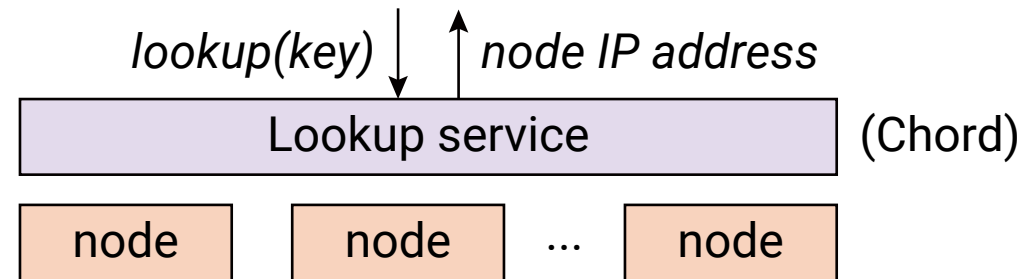
- Distributed Hash Table: an abstraction of hash table in a distributed setting
 - ▶ `lookup(key) -> IP addr` (**Chord lookup service**)
 - ▶ `send-RPC(IP address, put, key, data_one)`
 - ▶ `send-RPC(IP address, get, key) -> data_one`

What is a DHT (and Why)?

- Distributed Hash Table: an abstraction of hash table in a distributed setting
 - `lookup(key) -> IP addr` (**Chord lookup service**)
 - `send-RPC(IP address, put, key, data_one)`
 - `send-RPC(IP address, get, key) -> data_one`
- Partitioning data in **large-scale distributed systems**
 - Tuples in a global database engine
 - Data blocks in a global file system
 - Files in a P2P file-sharing system

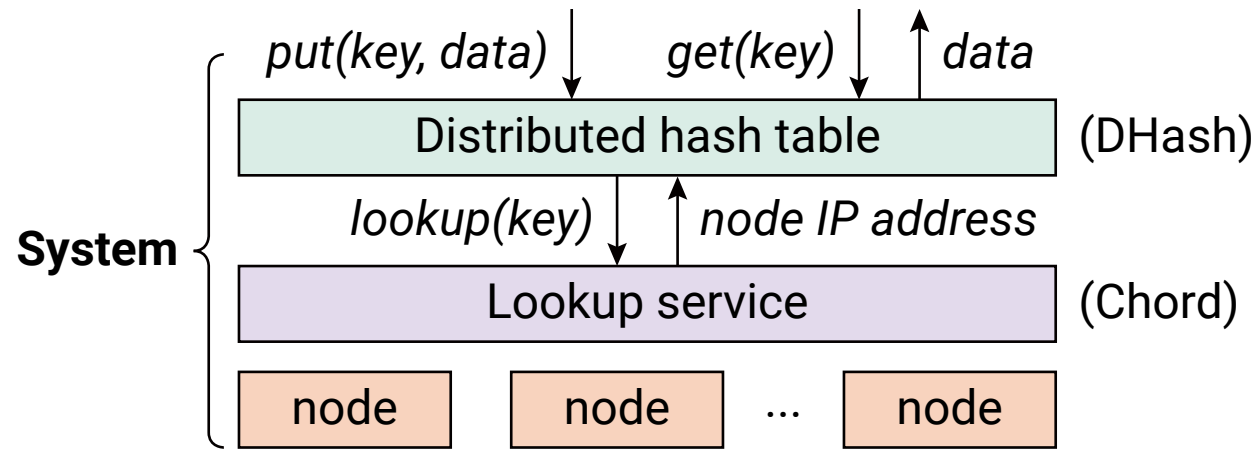


Cooperative Storage with a DHT

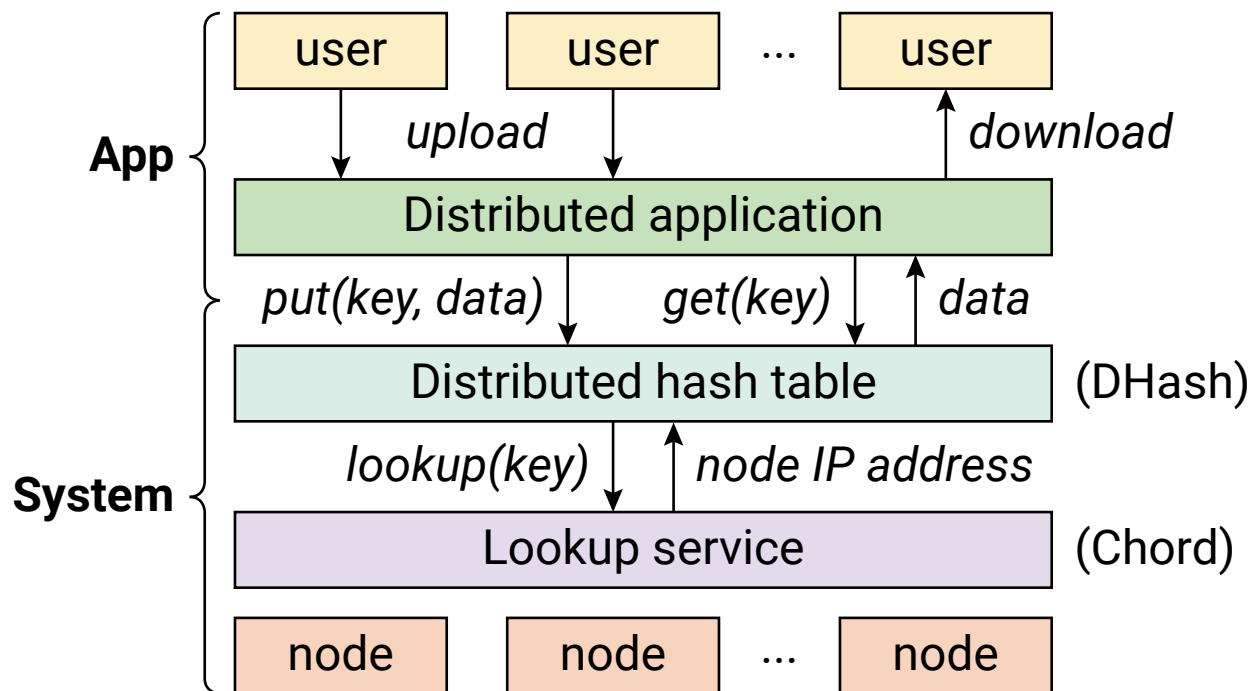




Cooperative Storage with a DHT



Cooperative Storage with a DHT



DHT is Expected to be



- **Scalability**: The system should handle millions of nodes.
- **Fault Tolerance**: The system must function even as nodes join and leave continuously.
- **Efficiency**: Finding a key should require minimal messages (logarithmic or better).
- **Decentralization**: No single node is “more important” than another; there is no single point of failure.

DHT is Expected to be

- **Scalability**: The system should handle millions of nodes.
- **Fault Tolerance**: The system must function even as nodes join and leave continuously.
- **Efficiency**: Finding a key should require minimal messages (logarithmic or better).
- **Decentralization**: No single node is “more important” than another; there is no single point of failure.

Challenge How do we decide which of the thousands of nodes stores which key-value pair without a central directory?

- Hashing is a natural solution, but how do we do it in a distributed way?

Why $\text{Hash}(\text{key})(\text{mod } N)$ Fails

- Concept: In a cluster of N nodes, assign key k to node $\text{Hash}(\text{key})(\text{mod } N)$
- Problem: Peer-to-peer systems are dynamic (Churn)
 - If a node joins, N becomes $N + 1$
 - If a node leaves, N becomes $N - 1$.
- When N changes, nearly every key in the system maps to a new node ($\text{Hash}(\text{key})(\text{mod } N) \neq \text{Hash}(\text{key})(\text{mod } N + 1)$).
 - Massive data movement across the network just to reorganize. This does not scale for P2P.

mod(n) hashing

- System of n nodes: $1 \dots n$
 - Node that owns key is assigned via $\text{hash}(\text{key}) \bmod n$
 - Good load balancing
- What if a node fails?
 - Instead of n nodes, now $n - 1$ nodes
 - Mapping of all keys change, as now $\text{hash}(\text{key}) \bmod (n - 1)$

mod(n) hashing

- System of n nodes: $1 \dots n$
 - Node that owns key is assigned via $\text{hash}(\text{key}) \bmod n$
 - Good load balancing
- What if a node fails?
 - Instead of n nodes, now $n - 1$ nodes
 - Mapping of all keys change, as now $\text{hash}(\text{key}) \bmod (n - 1)$
- $N = 5$
 - $12594 \bmod 5 = 4$
 - $28527 \bmod 5 = 2$
 - $816 \bmod 5 = 1$
 - $716565 \bmod 5 = 0$

mod(n) hashing

- System of n nodes: $1 \dots n$
 - Node that owns key is assigned via $\text{hash}(\text{key}) \bmod n$
 - Good load balancing
- What if a node fails?
 - Instead of n nodes, now $n - 1$ nodes
 - Mapping of all keys change, as now $\text{hash}(\text{key}) \bmod (n - 1)$
- $N = 5$
 - $12594 \bmod 5 = 4$
 - $28527 \bmod 5 = 2$
 - $816 \bmod 5 = 1$
 - $716565 \bmod 5 = 0$
- $N = 4$
 - $12594 \bmod 4 = 2$
 - $28527 \bmod 4 = 3$
 - $816 \bmod 4 = 0$
 - $716565 \bmod 4 = 1$

► Solution: Consistent Hashing

- **Objective:** Minimize the number of keys that need to be remapped when the number of nodes changes
- **Goal:** When a node joins or leaves, only K/N keys should move
 - K is the total number of keys
- **Strategy:** Decouple the keys from the number of nodes.
 - Instead of mapping to a fixed set of slots, map both keys and nodes into a large, shared “Identifier Space”

Chord: A DHT Protocol

- Nodes and keys are assigned an m -bit identifier using **consistent hashing**.
 - Hashed values (integers) using the same hash function
 - Key identifier = $\text{SHA-1}(\text{key}) \bmod 2^{160}$
 - Node identifier = $\text{SHA-1}(\text{IP address}) \bmod 2^{160}$

- What is “SHA-1”?
 - SHA-1 is a cryptographic hash function that maps input to 160-bit output hash
 - Some properties:
 1. Output hashes looks randomly distributed across output space
 2. Given `hash1`, hard to find `input1` where $\text{SHA-1}(\text{input1}) = \text{hash1}$
 3. Given `input1` and `hash1`, hard to find `input2` where $\text{SHA-1}(\text{input2}) = \text{hash1}$
 4. Hard to find `input1` and `input2` where $\text{SHA-1}(\text{input1}) = \text{SHA-1}(\text{input2})$

Chord Identifiers

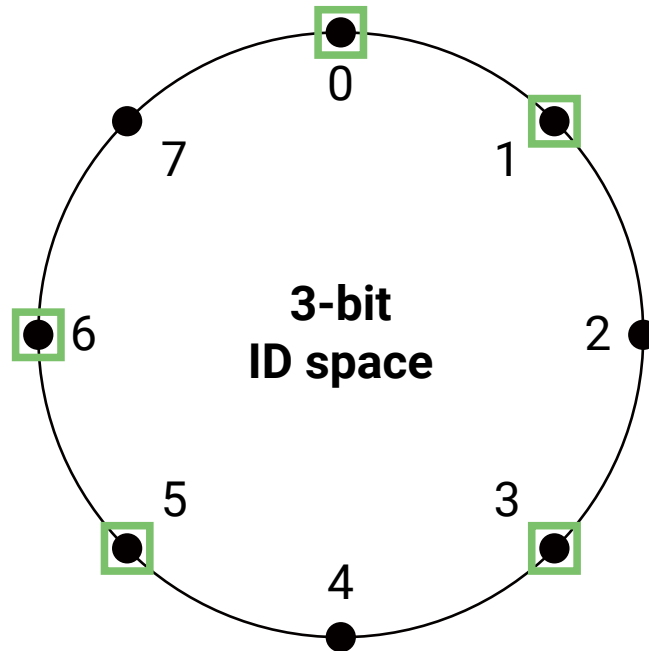
- Nodes and keys are assigned an m -bit identifier using **consistent hashing**.
 - Key identifier = $\text{SHA-1}(\text{key}) \bmod 2^{160}$
 - Node identifier = $\text{SHA-1}(\text{IP address}) \bmod 2^{160}$
- How does Chord partition data?
 - i.e., map key IDs to node IDs
- Why hash key and address?
 - Uniformly distributed in the ID space
 - Hashed key $\rightarrow$ load balancing; hashed address $\rightarrow$ independent failure

Consistent Hashing: Data Partitioning

- Identifiers have $m = 3$ bits
- Key space: $[0, 2^3 - 1]$

● Identifiers/key space

□ Node



Consistent Hashing: Data Partitioning

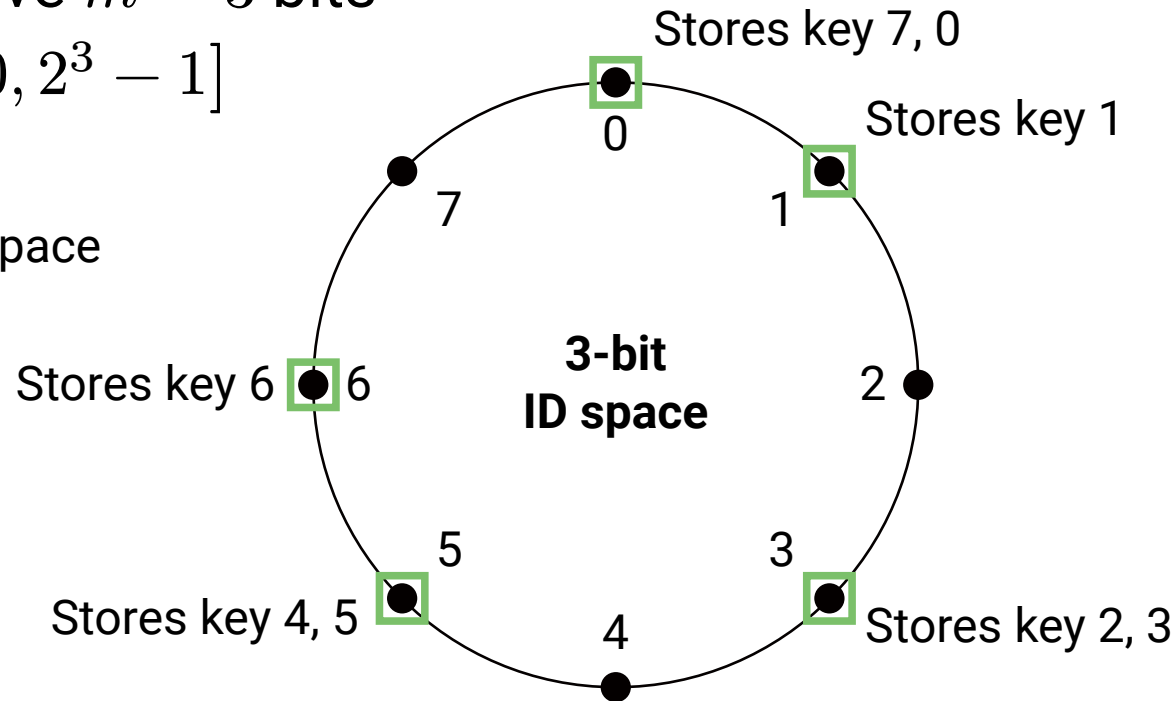
Successor Rule A key k is assigned to the first node whose ID is equal to or follows ID_k in the identifier space

- Terminology: This node is called the **successor** of key k
 - If keys are at positions 10, 25, and 50
 - And nodes are at positions 20 and 100
 - Key **10** goes to Node **20**
 - Key **25** and Key **50** go to Node **100**

Consistent Hashing: Data Partitioning

- Identifiers have $m = 3$ bits
- Key space: $[0, 2^3 - 1]$

● Identifiers/key space
□ Node

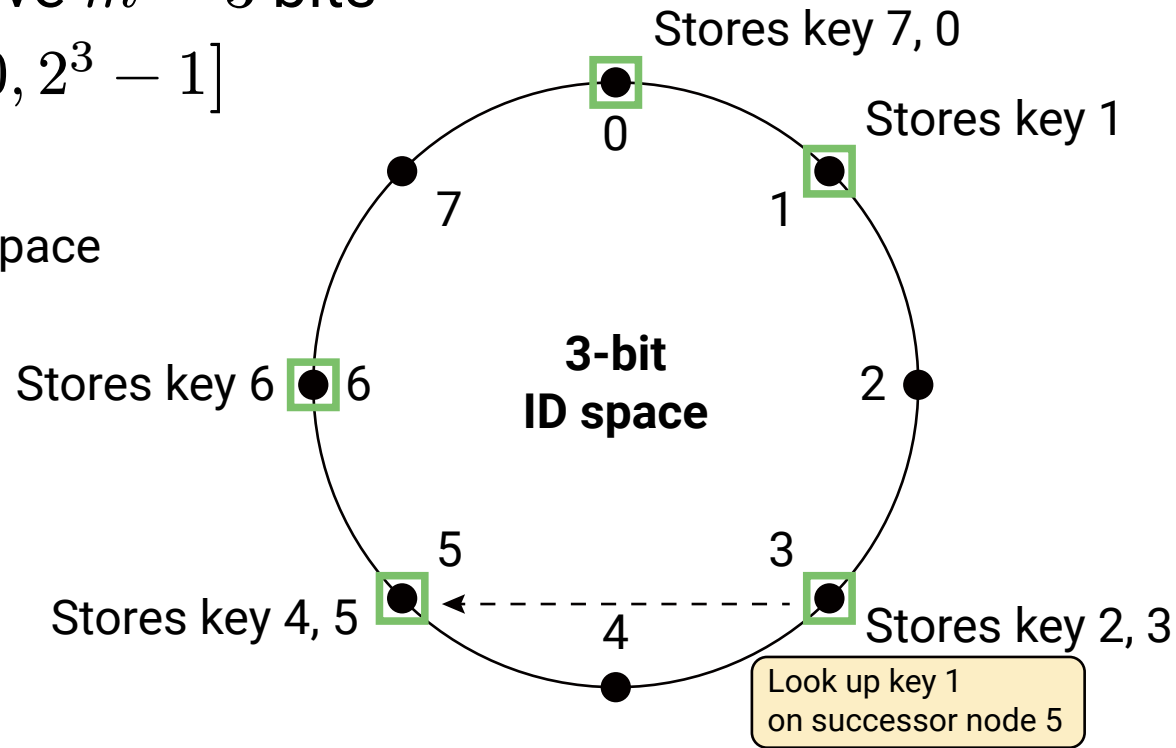


Key is stored at its **successor**: node with next-higher ID

Strawman Lookup via Successors

- Identifiers have $m = 3$ bits
- Key space: $[0, 2^3 - 1]$

● Identifiers/key space
 □ Node

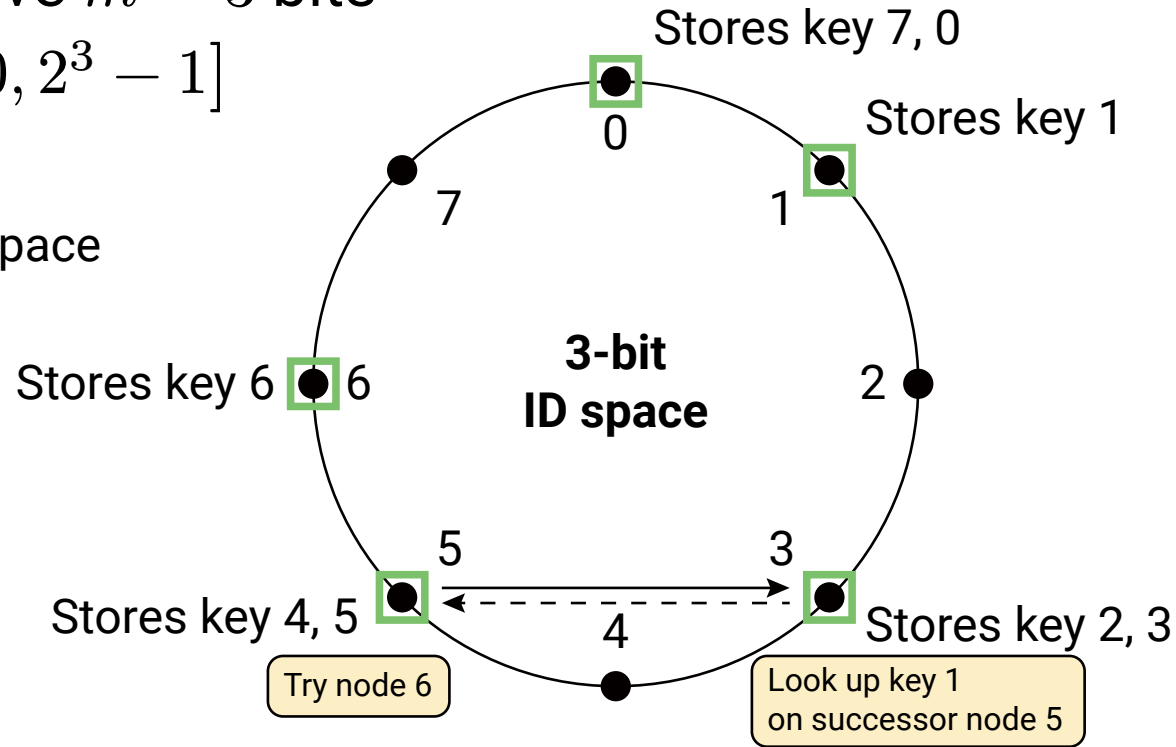


Strawman Lookup via Successors

- Identifiers have $m = 3$ bits
- Key space: $[0, 2^3 - 1]$

● Identifiers/key space

□ Node

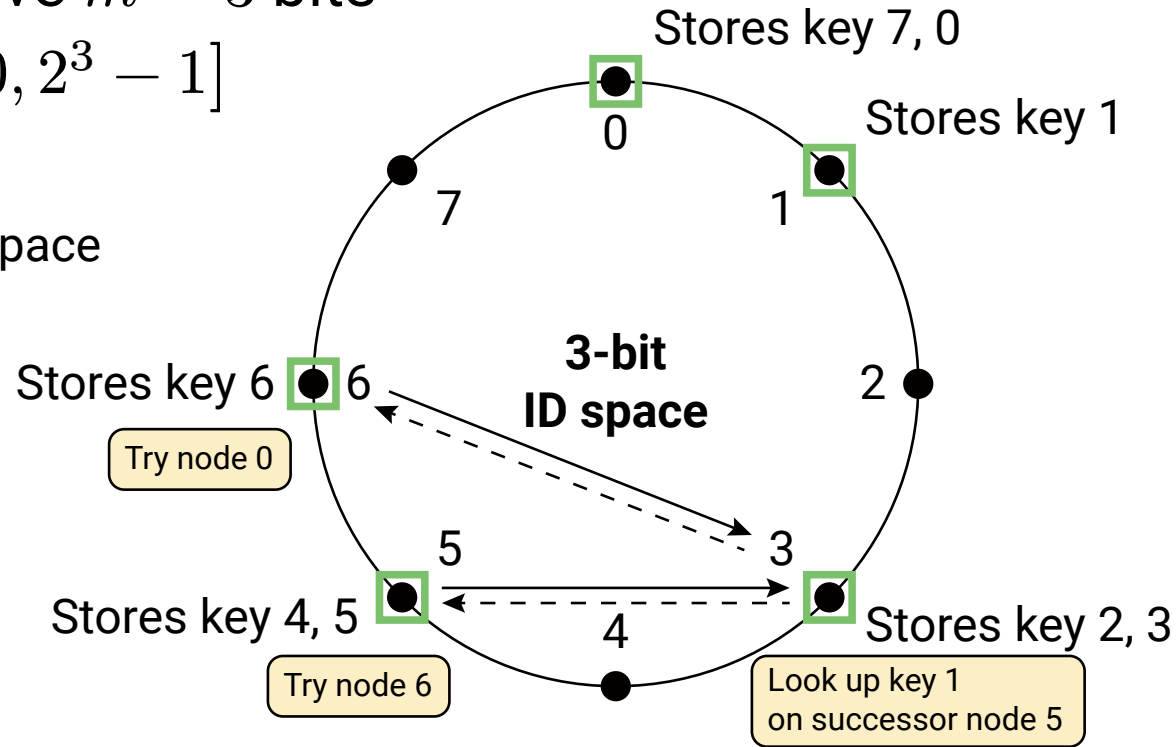


Strawman Lookup via Successors

- Identifiers have $m = 3$ bits
- Key space: $[0, 2^3 - 1]$

● Identifiers/key space

□ Node

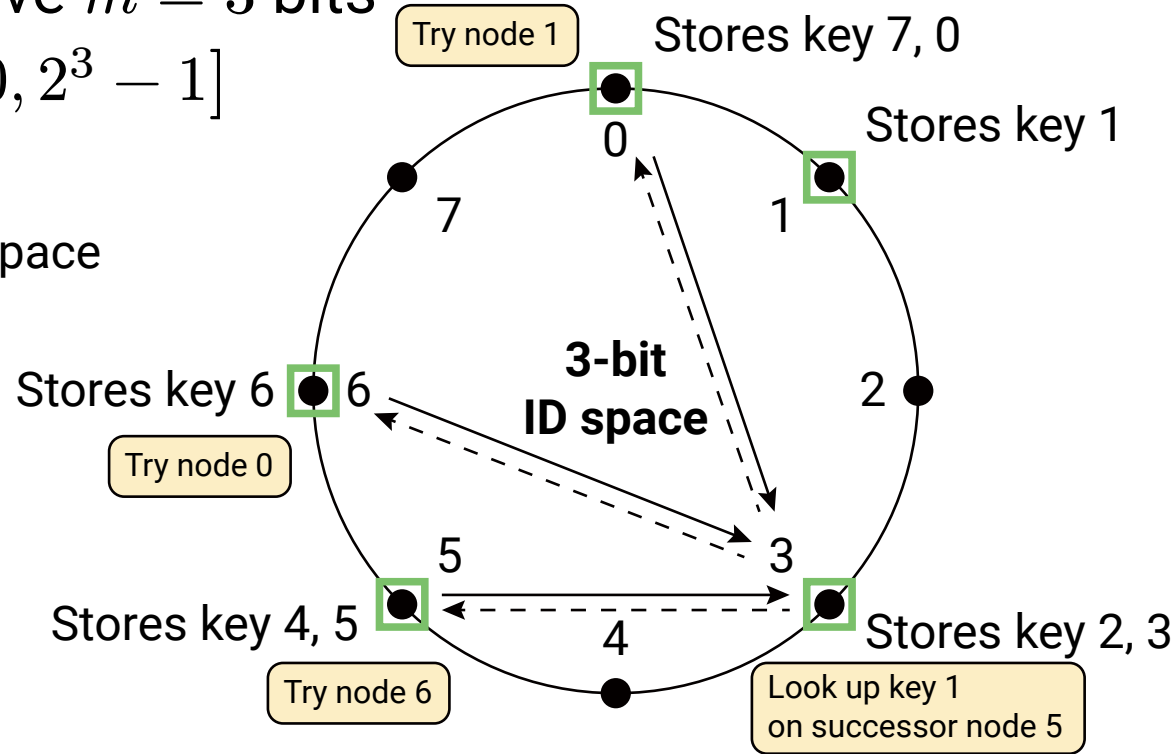


Strawman Lookup via Successors

- Identifiers have $m = 3$ bits
- Key space: $[0, 2^3 - 1]$

● Identifiers/key space

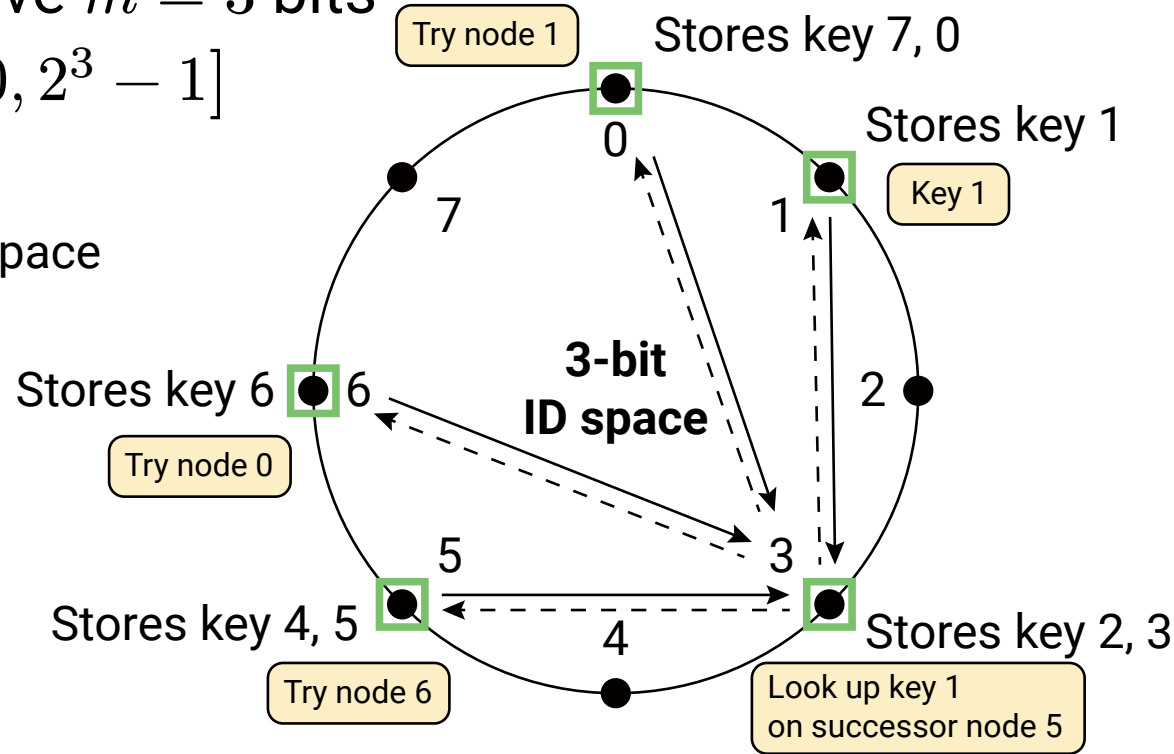
□ Node



Strawman Lookup via Successors

- Identifiers have $m = 3$ bits
- Key space: $[0, 2^3 - 1]$

● Identifiers/key space
 □ Node



$O(N)$ messages and hops!

Beyond the Neighbor: Why We Need Routing

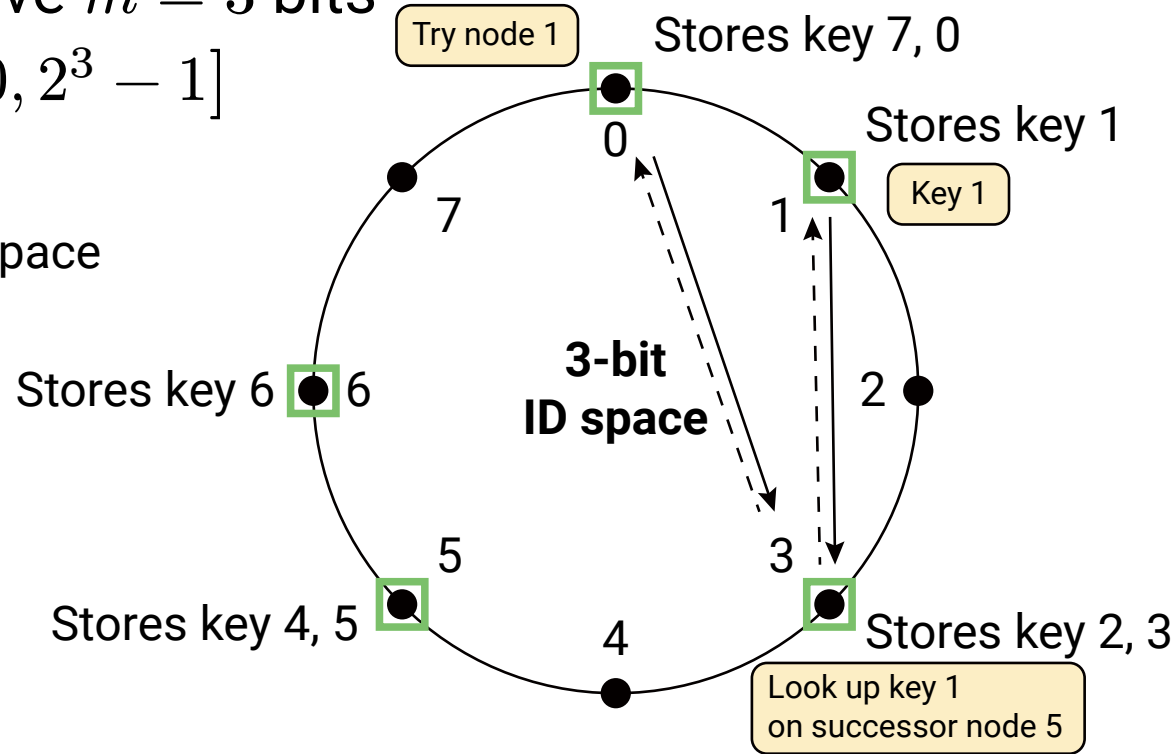
- We have a ring of N nodes
 - Each node knows its immediate successor
- **The Linear Approach:** To find a key, we could just pass the request from neighbor to neighbor
 - **Pros:** Simple, requires only 1 pointer per node
 - **Cons:** Inefficient. For N nodes, it takes $O(N)$ hops on average to find a key

Goal We need a way to “jump” across the ring to reach the destination faster

Observation About Last Hop

- Identifiers have $m = 3$ bits
- Key space: $[0, 2^3 - 1]$

● Identifiers/key space
 □ Node



Try to find key's **predecessor node** as fast as possible. This predecessor will know key's successor (owner).



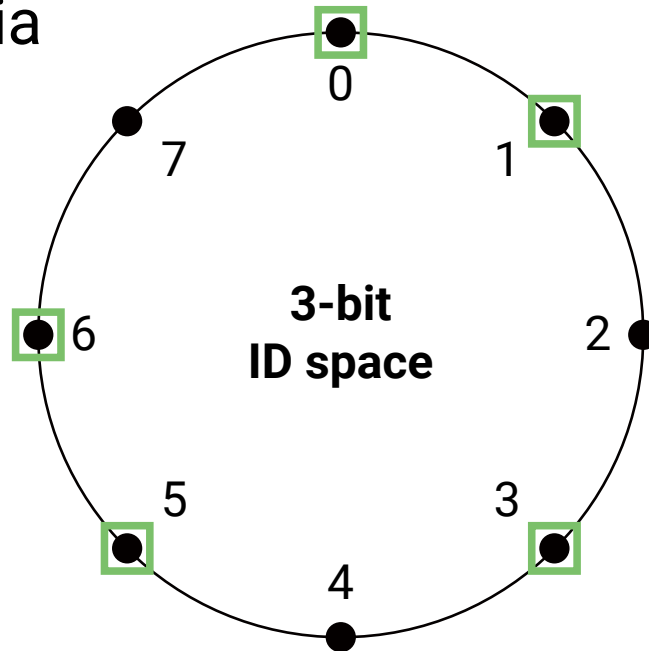
Chord — Finger Tables for **find_predecessor**



- Each node n maintains a routing table called a **Finger Table** with at most m entries
 - m is the number of bits in the ID space
- The i^{th} entry in the table at node n contains the identity of the first node s that succeeds n by at least 2^{i-1} on the identifier circle
- Formula: $\text{finger}[i] = \text{successor}((n + 2^{i-1})(\text{mod}\{2^m\}))$
 - Each node knows nodes at exponential distances (1, 2, 4, 8, ...) ahead of it.

Chord – Finger Tables for **find_predecessor**

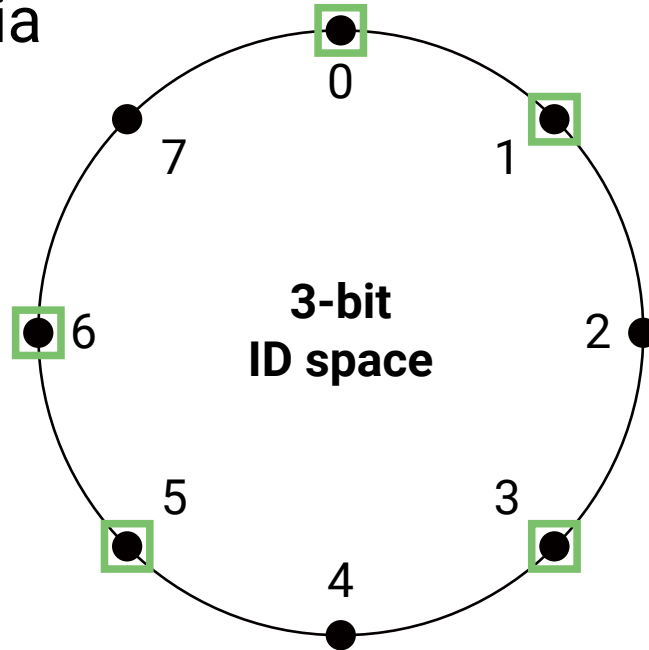
- Each node keeps m states
- Key space: m ranges via
 - $(N + 2^{k-1}) \bmod 2^m$
 - $1 \leq k \leq m$



Chord – Finger Tables for **find_predecessor**

- Each node keeps m states
- Key space: m ranges via
 - $(N + 2^{k-1}) \bmod 2^m$
 - $1 \leq k \leq m$

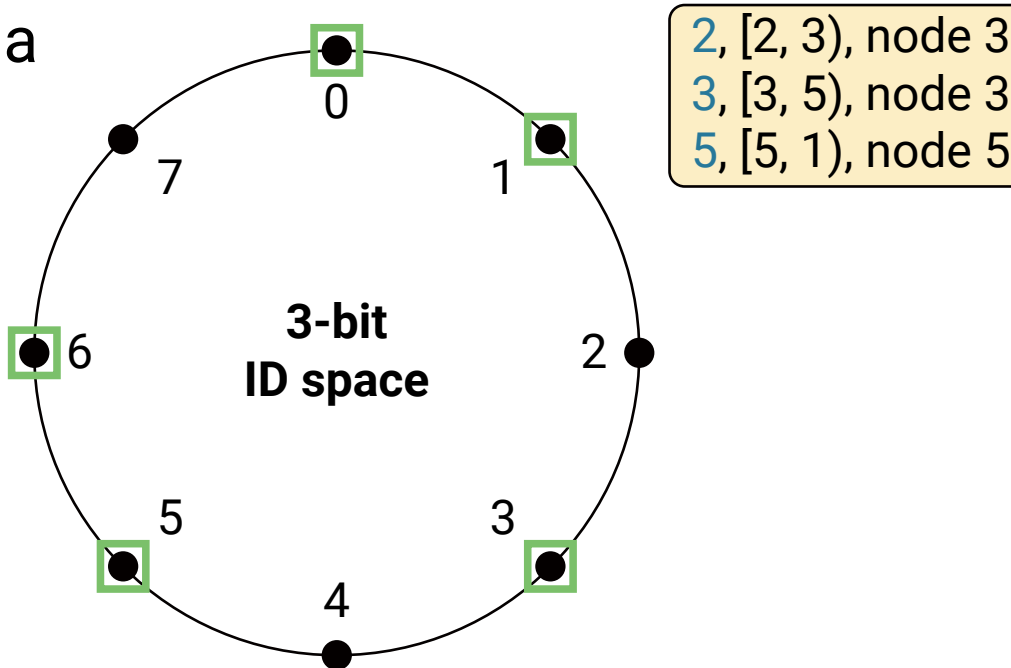
- Example: node $N = 1$
 - $1 + 1 \bmod 8 = 2$
 - $1 + 2 \bmod 8 = 3$
 - $1 + 4 \bmod 8 = 5$



Chord – Finger Tables for **find_predecessor**

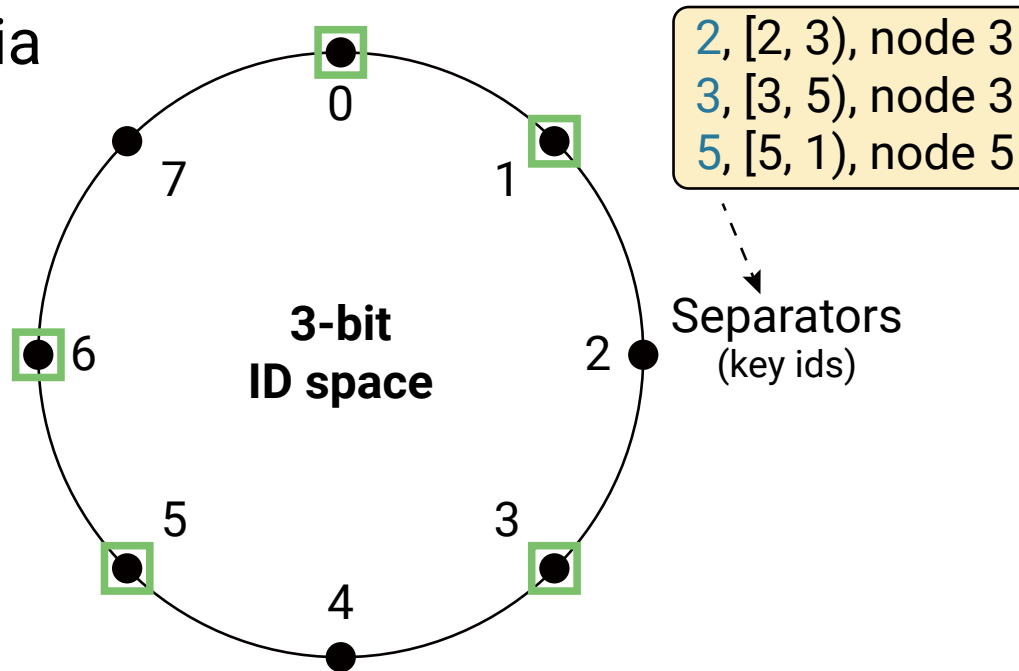
- Each node keeps m states
- Key space: m ranges via
 - $(N + 2^{k-1}) \bmod 2^m$
 - $1 \leq k \leq m$

- Example: node $N = 1$
 - $1 + 1 \bmod 8 = 2$
 - $1 + 2 \bmod 8 = 3$
 - $1 + 4 \bmod 8 = 5$



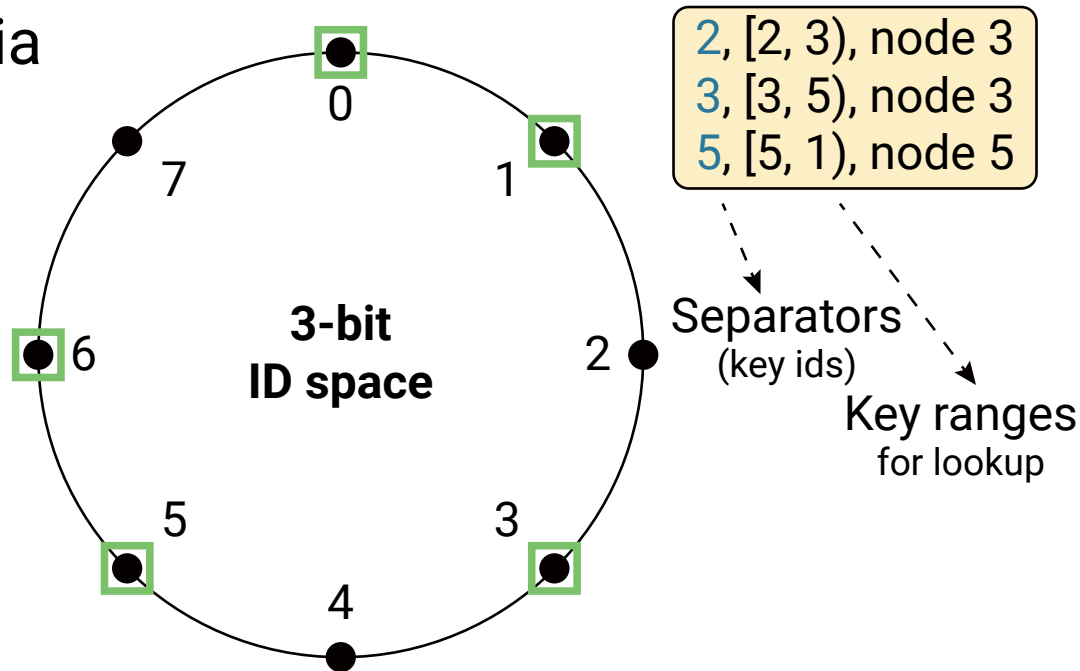
Chord – Finger Tables for **find_predecessor**

- Each node keeps m states
- Key space: m ranges via
 - $(N + 2^{k-1}) \bmod 2^m$
 - $1 \leq k \leq m$
- Example: node $N = 1$
 - $1 + 1 \bmod 8 = 2$
 - $1 + 2 \bmod 8 = 3$
 - $1 + 4 \bmod 8 = 5$



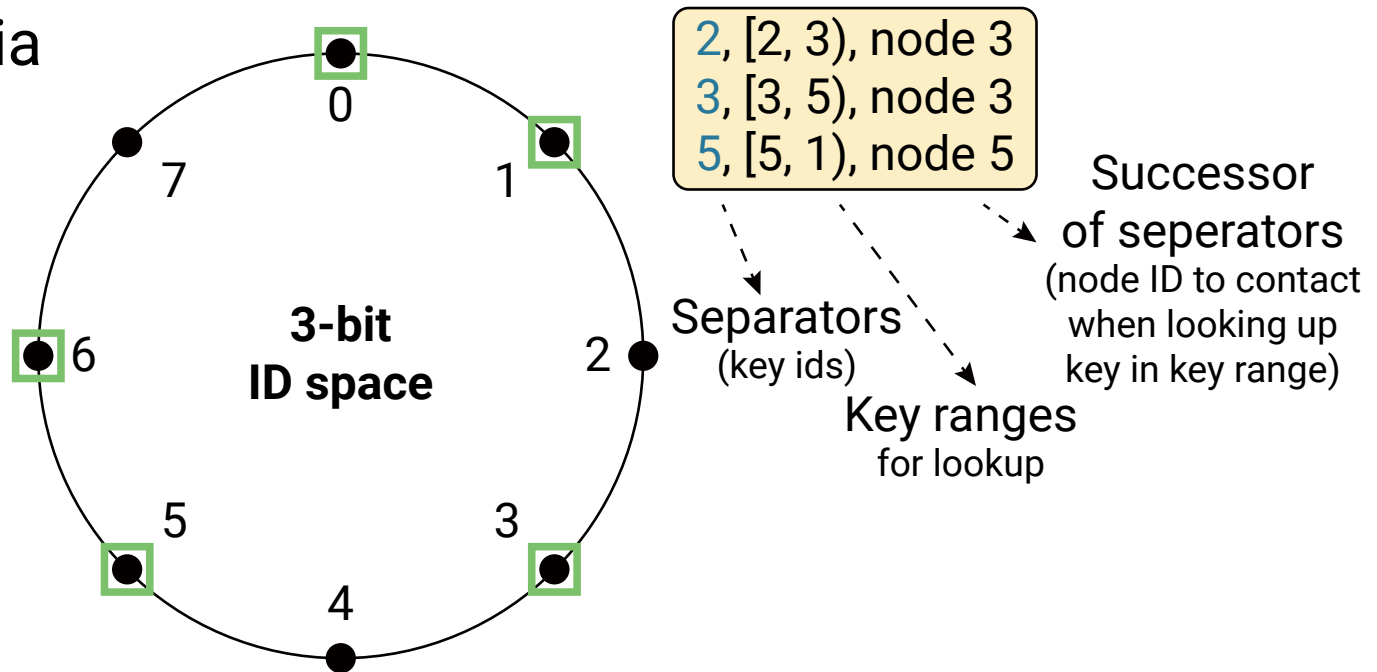
Chord – Finger Tables for **find_predecessor**

- Each node keeps m states
- Key space: m ranges via
 - $(N + 2^{k-1}) \bmod 2^m$
 - $1 \leq k \leq m$
- Example: node $N = 1$
 - $1 + 1 \bmod 8 = 2$
 - $1 + 2 \bmod 8 = 3$
 - $1 + 4 \bmod 8 = 5$



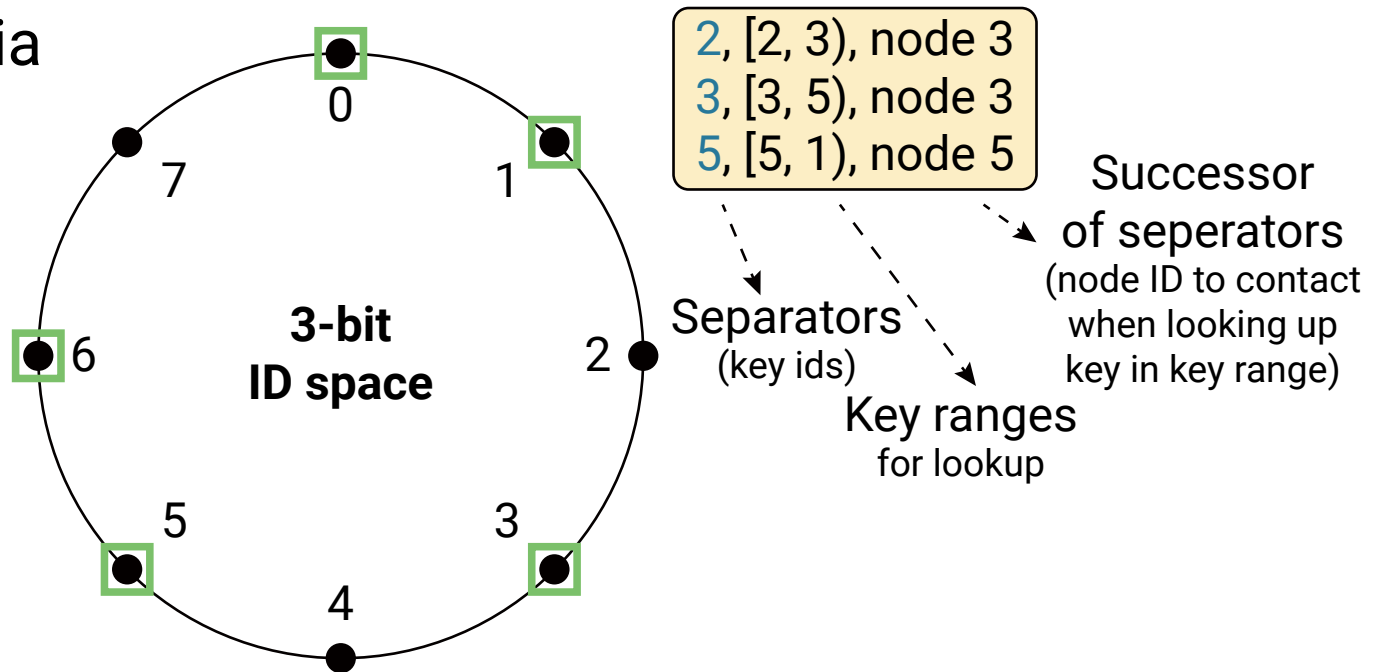
Chord – Finger Tables for **find_predecessor**

- Each node keeps m states
- Key space: m ranges via
 - $(N + 2^{k-1}) \bmod 2^m$
 - $1 \leq k \leq m$
- Example: node $N = 1$
 - $1 + 1 \bmod 8 = 2$
 - $1 + 2 \bmod 8 = 3$
 - $1 + 4 \bmod 8 = 5$



Chord – Finger Tables for **find_predecessor**

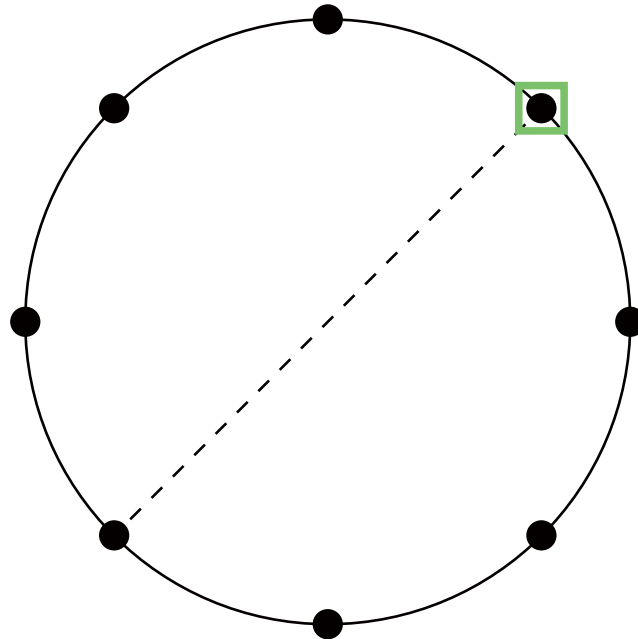
- Each node keeps m states
- Key space: m ranges via
 - $(N + 2^{k-1}) \bmod 2^m$
 - $1 \leq k \leq m$
- Example: node $N = 1$
 - $1 + 1 \bmod 8 = 2$
 - $1 + 2 \bmod 8 = 3$
 - $1 + 4 \bmod 8 = 5$



“Finger” is the node immediately succeeding separator

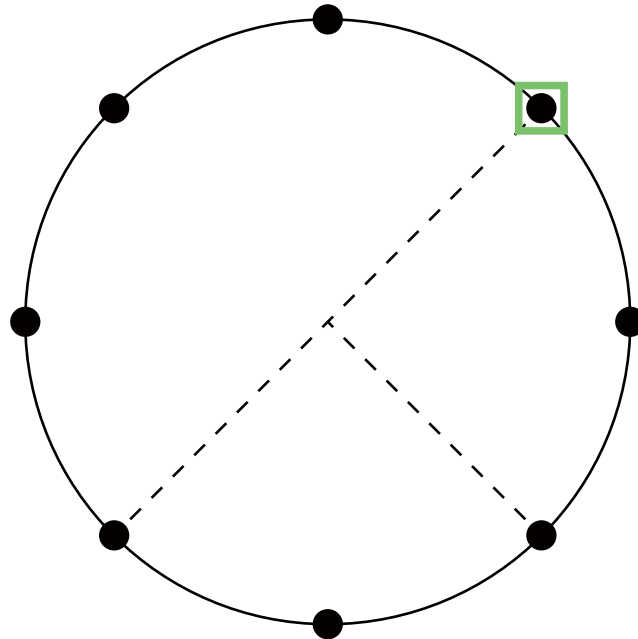


Chord — Finger Tables for **find_predecessor**



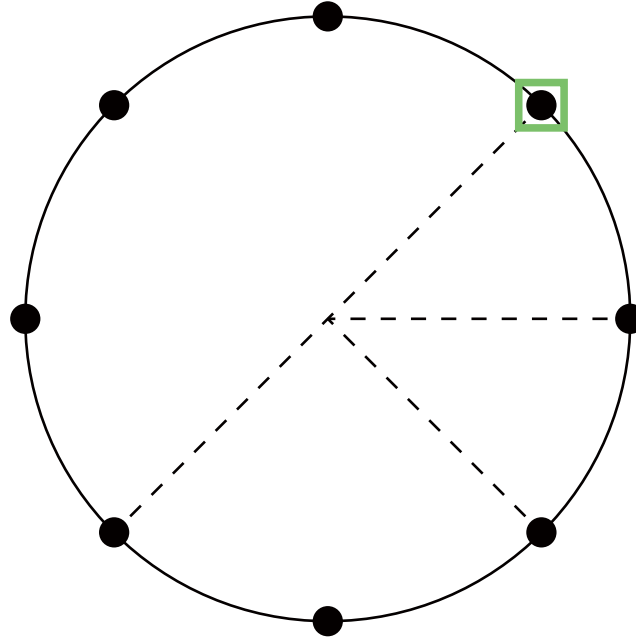


Chord — Finger Tables for **find_predecessor**



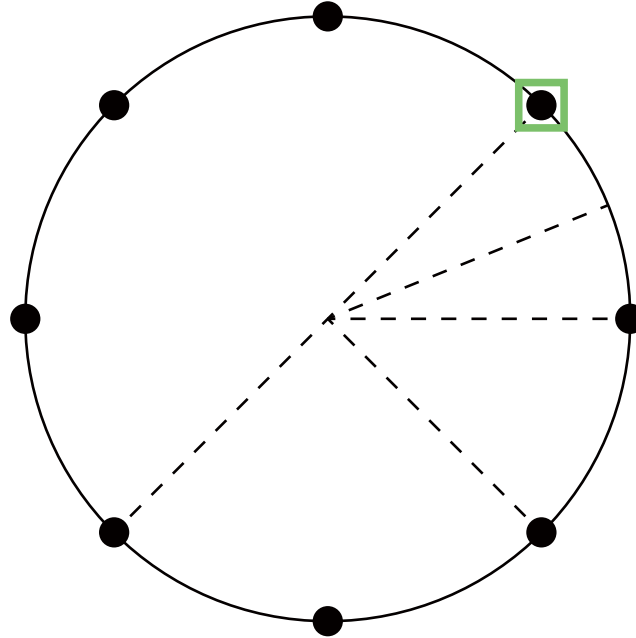


Chord – Finger Tables for **find_predecessor**



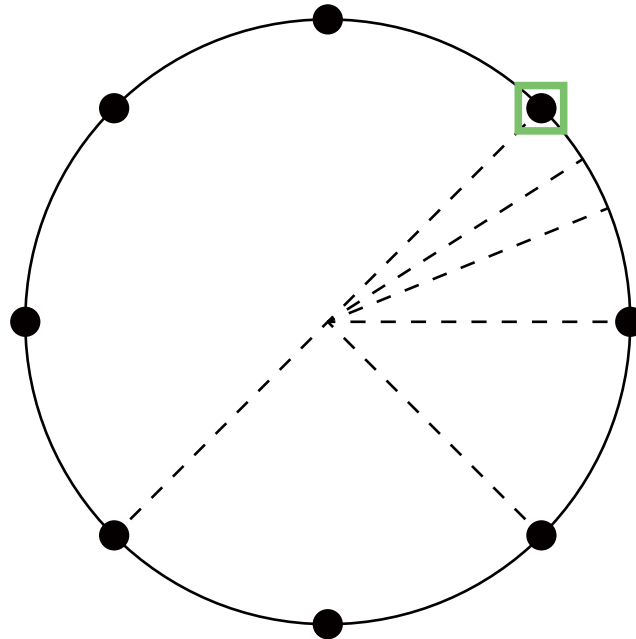


Chord – Finger Tables for **find_predecessor**



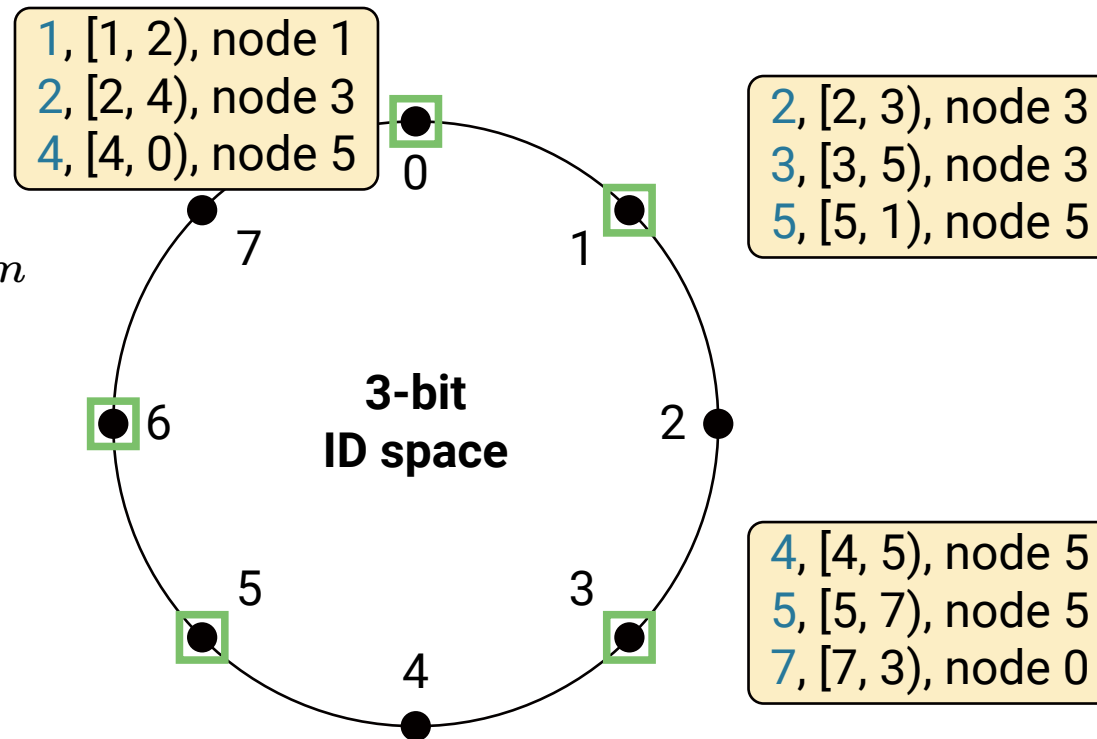


Chord — Finger Tables for **find_predecessor**



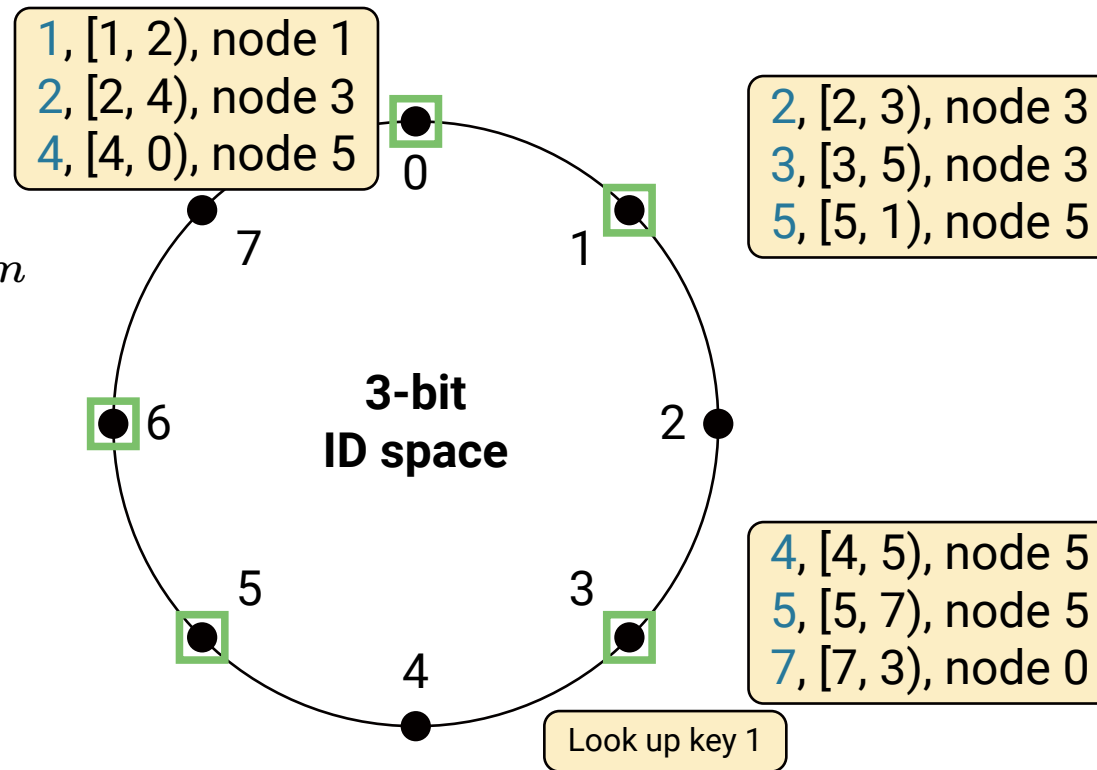
Chord – Finger Tables for **find_predecessor**

- Node: m states
- Key space:
 - m ranges via
 - $(N + 2^{k-1}) \bmod 2^m$
 - $1 \leq k \leq m$



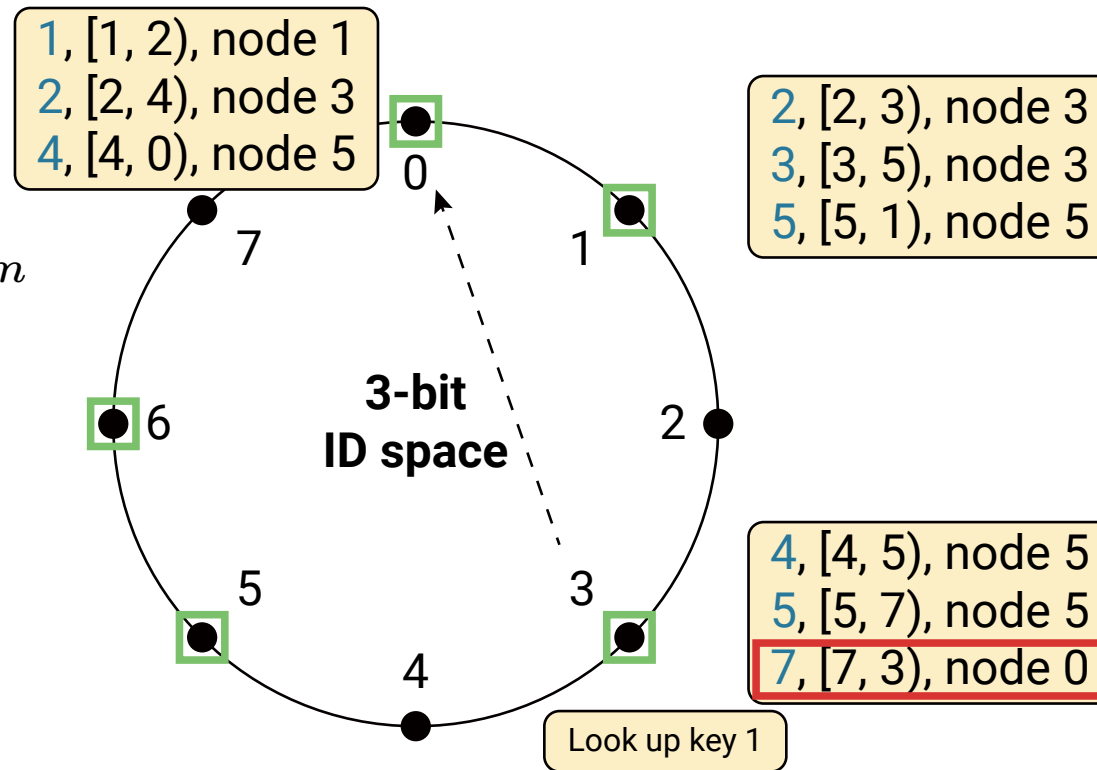
Chord – Finger Tables for **find_predecessor**

- Node: m states
- Key space:
 - m ranges via
 - $(N + 2^{k-1}) \bmod 2^m$
 - $1 \leq k \leq m$



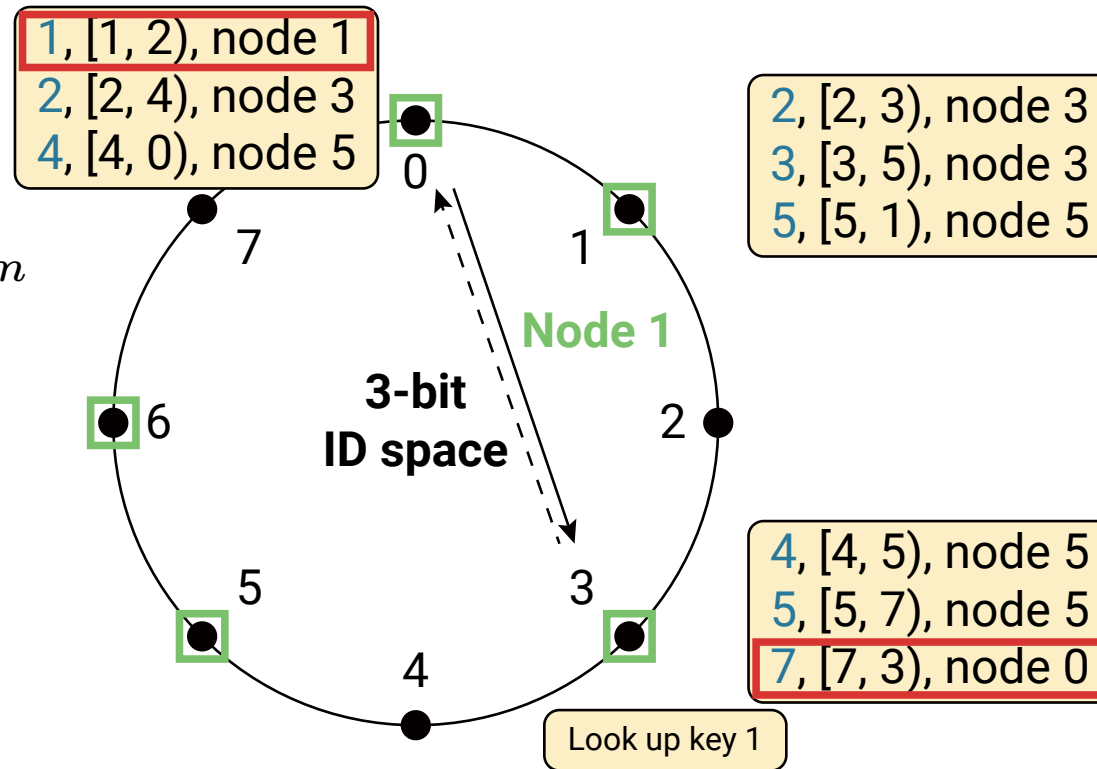
Chord – Finger Tables for **find_predecessor**

- Node: m states
- Key space:
 - m ranges via
 - $(N + 2^{k-1}) \bmod 2^m$
 - $1 \leq k \leq m$



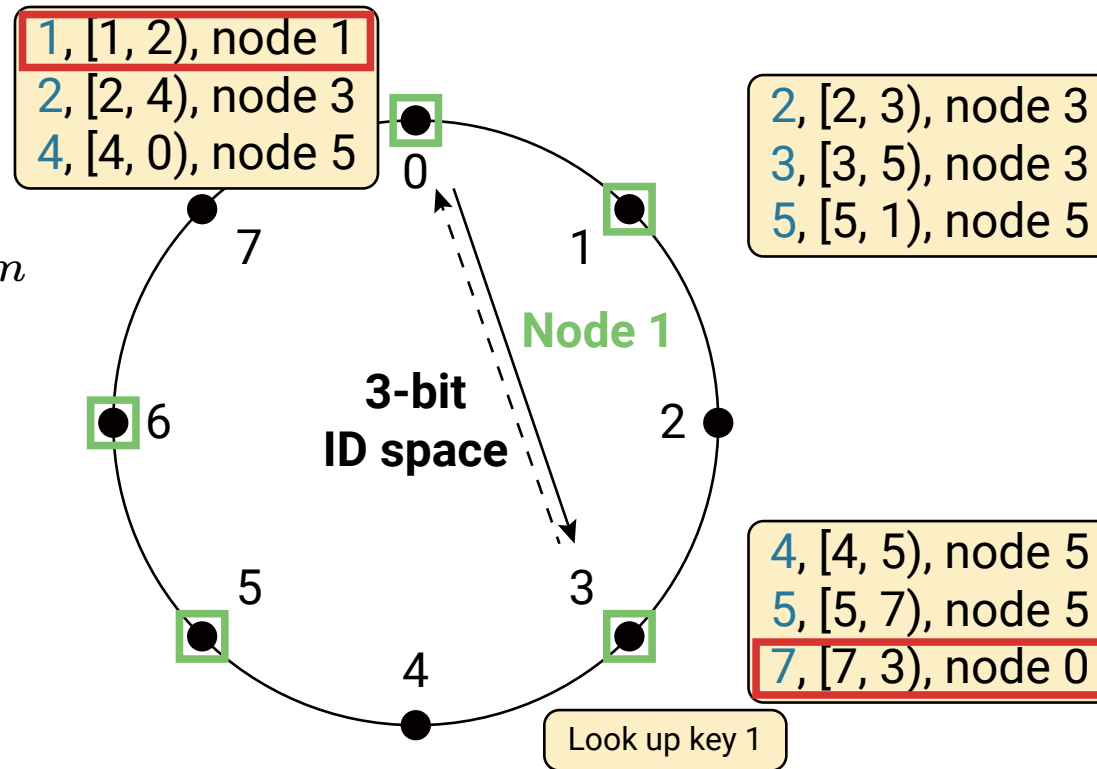
Chord – Finger Tables for **find_predecessor**

- Node: m states
- Key space:
 - m ranges via
 - $(N + 2^{k-1}) \bmod 2^m$
 - $1 \leq k \leq m$



Chord – Finger Tables for **find_predecessor**

- Node: m states
- Key space:
 - m ranges via
 - $(N + 2^{k-1}) \bmod 2^m$
 - $1 \leq k \leq m$



$O(\log N)$ messages and hops!

Implication of Finger Tables

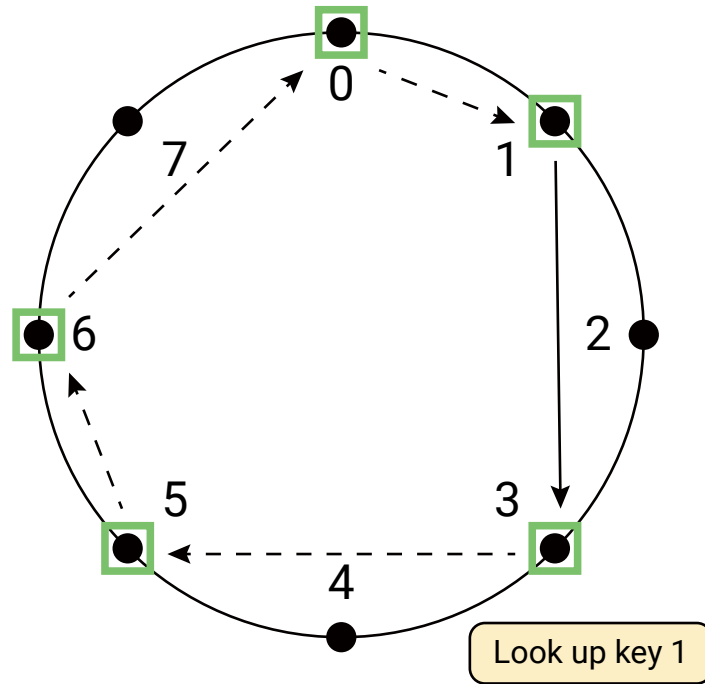
- A **binary lookup tree** rooted at every node
 - Threaded through other nodes' finger tables
- Better than arranging nodes in a single tree
 - Every node acts as a root
 - So there's **no root hotspot**
 - **No single point** of failure
 - But a lot **more state in total**: N nodes each have $O(\log N)$

Chord Lookup Algorithm Properties

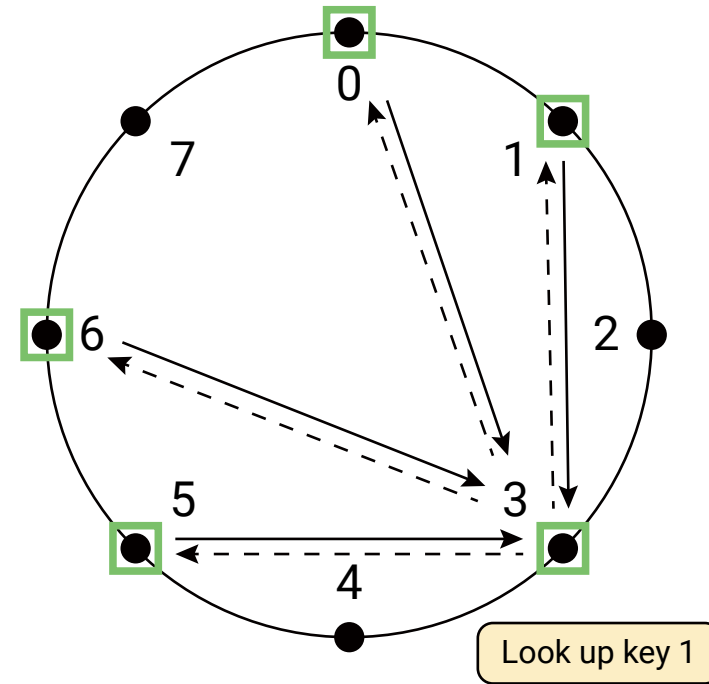
- **Interface:** $\text{lookup}(\text{key}) \rightarrow \text{IP address}$
- **Efficient:** $O(\log N)$ messages per lookup
 - N is the total number of nodes (peers)
- **Scalable:** $O(\log N)$ state per node
 - If $N = 1,000$, lookup takes ≈ 10 hops
 - If $N = 1,000,000$, lookup takes ≈ 20 hops
 - Increasing the network by $1000 \times$ only doubles the lookup time.
- **Robust:** survives massive failures



Chord – Recursive vs. Iterative Lookup



Recursive Lookup



Iterative Lookup

- Handling node **joins**
- Handling node **failures**
 1. Rebuilding lookup structures
 2. Ensure data durability

Chord — Node Joining

- A new node n must do three things to join:
 1. **Find its place**: n contacts an existing node and asks it to find $\text{successor}(n)$
 2. **Set its pointers**: n sets its successor and asks the successor for its current predecessor
 3. **Initialize its Finger Table**: n performs lookups to find the nodes responsible for its m finger intervals
- At this point, the node is in the ring, but other nodes don't know it yet.

Chord — Node Joining



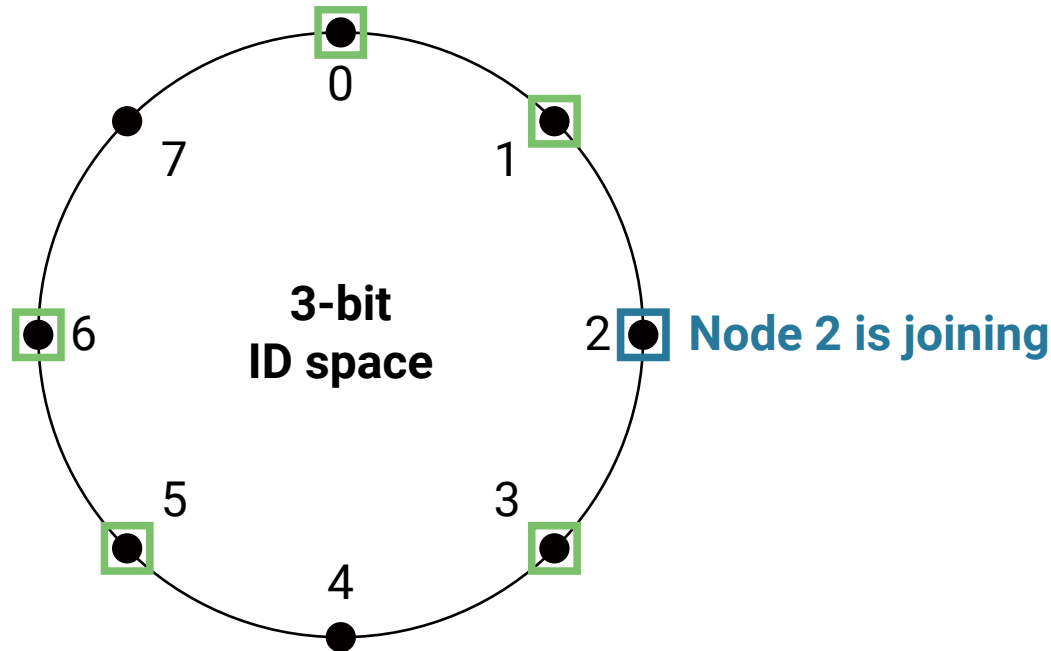
- **Handover**: A new node n only “takes” keys from its immediate successor
- Any keys k currently held by $\text{successor}(n)$ such that $\text{ID}_k \leq \text{ID}_n$ must be moved to n
 - Only the keys in the interval $(\text{predecessor}(n), n]$ move
 - No other nodes in the system are required to move data

Chord – Node Joining

- Identifiers have $m = 3$ bits
- Key space: $[0, 2^3 - 1]$

● Identifiers/key space

□ Node

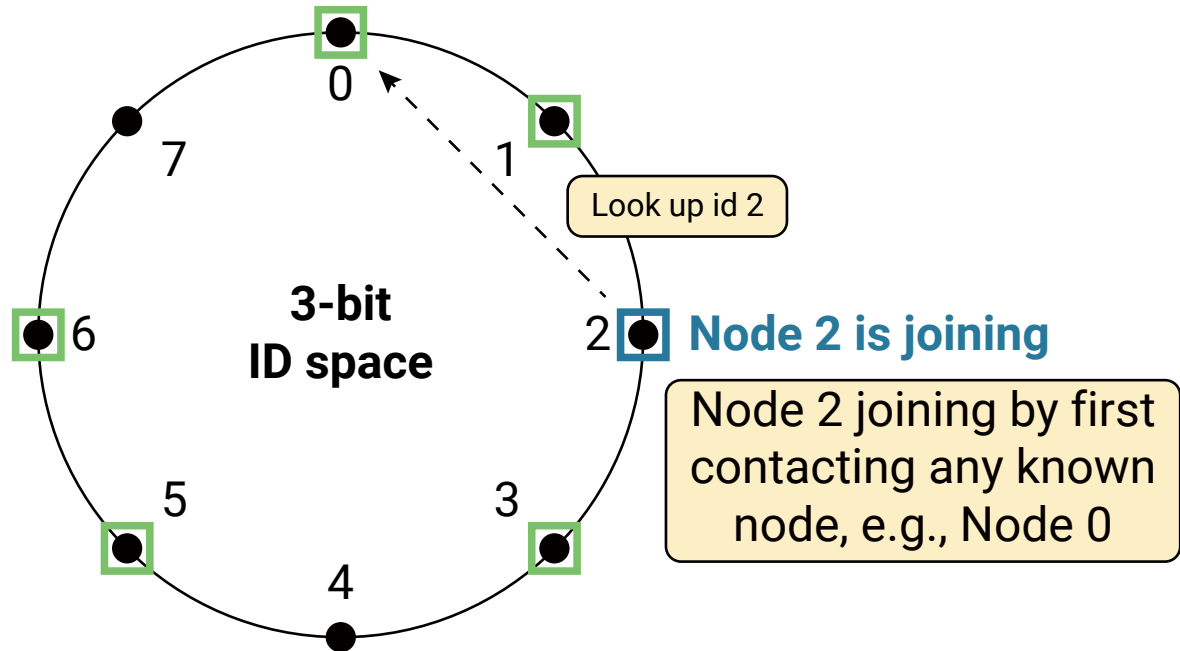


Chord – Node Joining

- Identifiers have $m = 3$ bits
- Key space: $[0, 2^3 - 1]$

● Identifiers/key space

□ Node

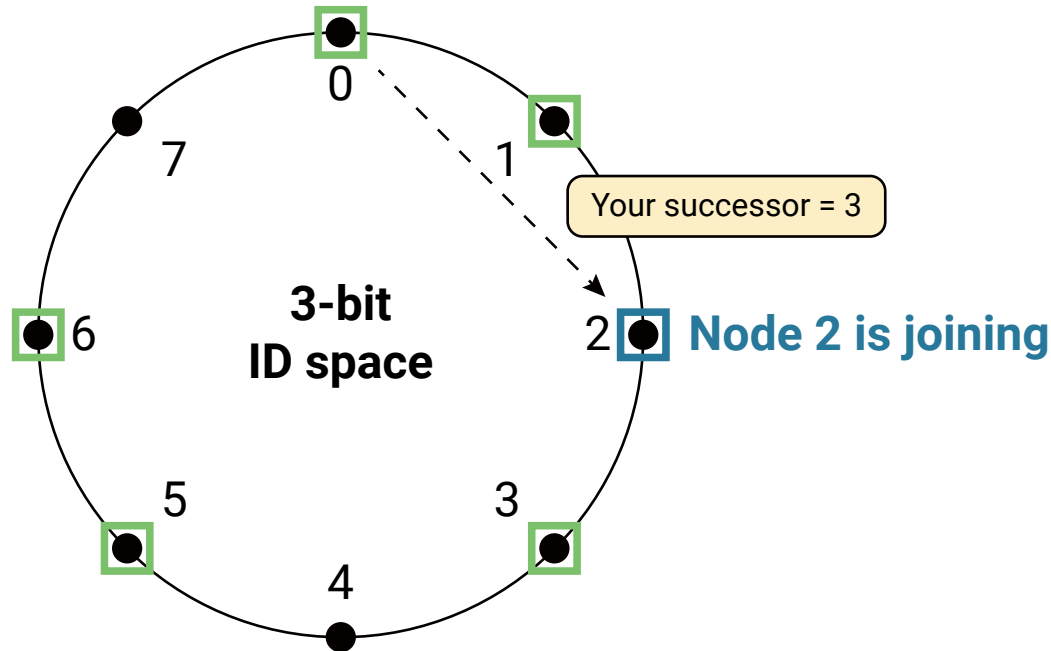


Chord – Node Joining

- Identifiers have $m = 3$ bits
- Key space: $[0, 2^3 - 1]$

● Identifiers/key space

□ Node

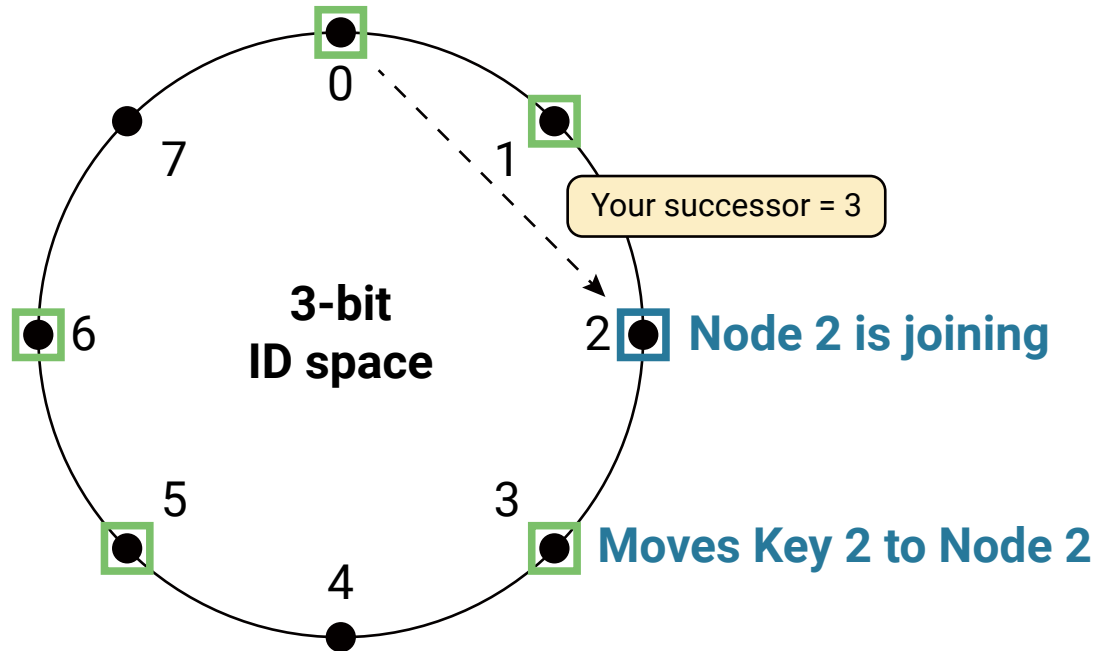


Chord – Node Joining

- Identifiers have $m = 3$ bits
- Key space: $[0, 2^3 - 1]$

● Identifiers/key space

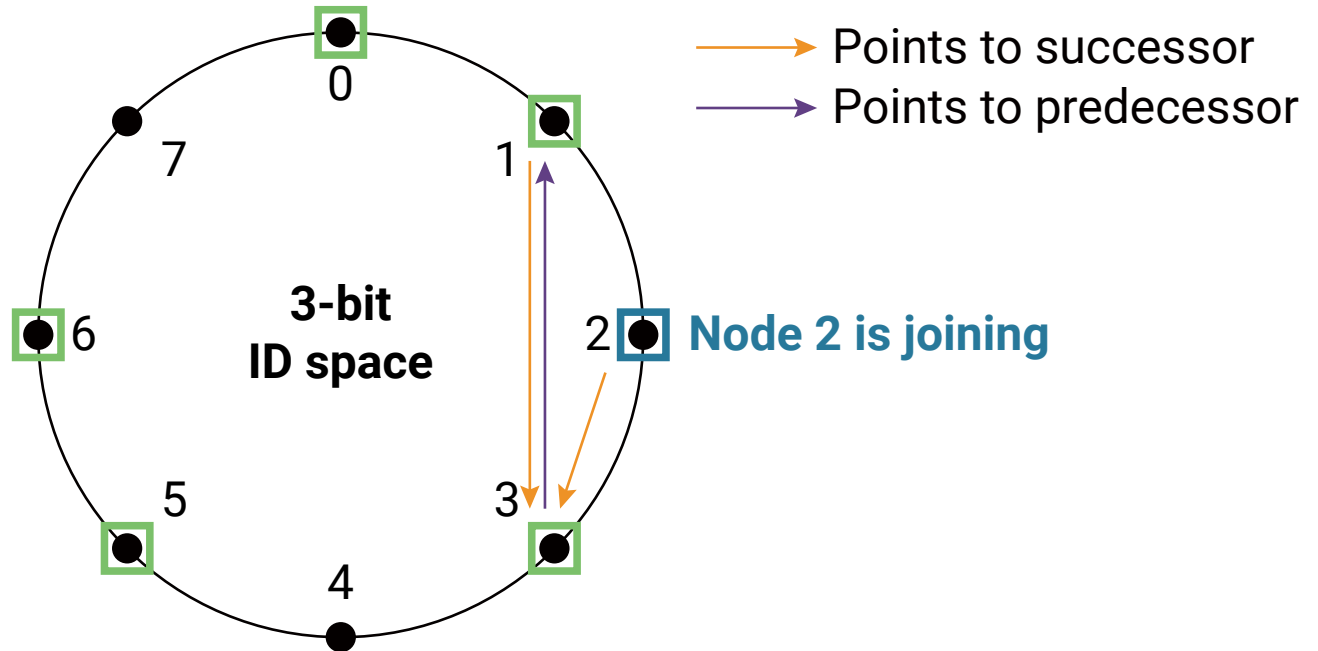
□ Node



Chord – Node Joining

- Identifiers have $m = 3$ bits
- Key space: $[0, 2^3 - 1]$

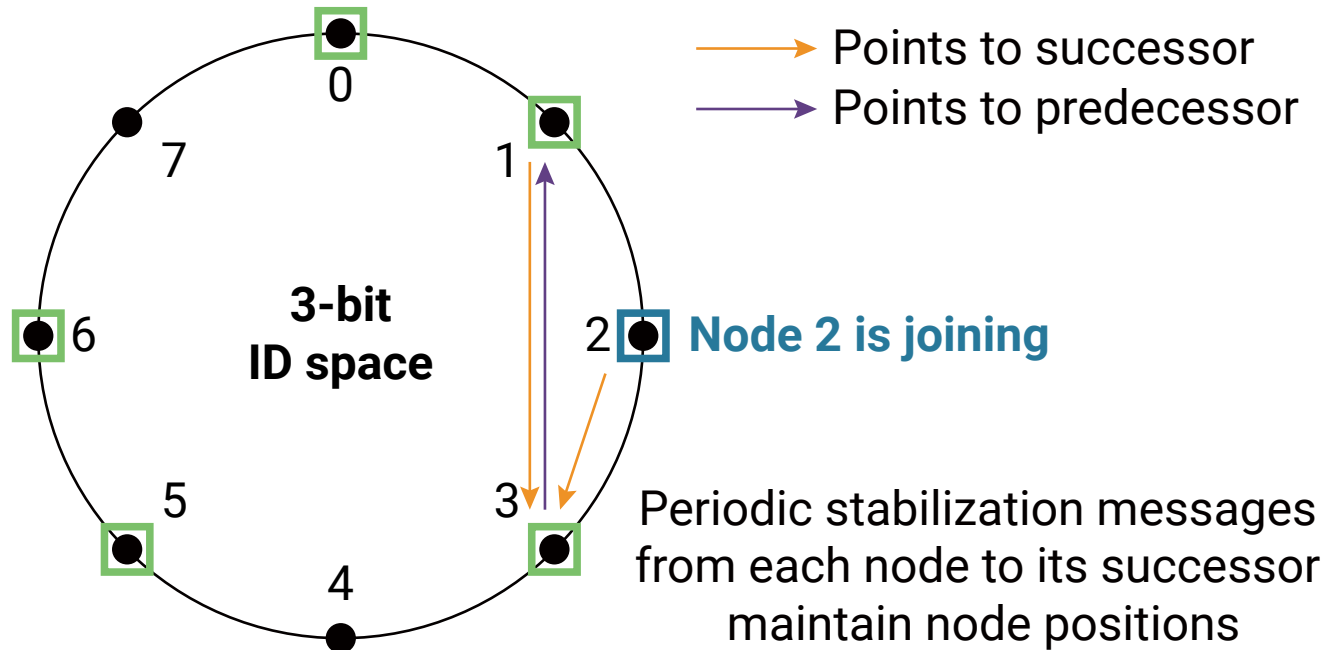
● Identifiers/key space
□ Node



Chord – Node Joining

- Identifiers have $m = 3$ bits
- Key space: $[0, 2^3 - 1]$

● Identifiers/key space
 □ Node

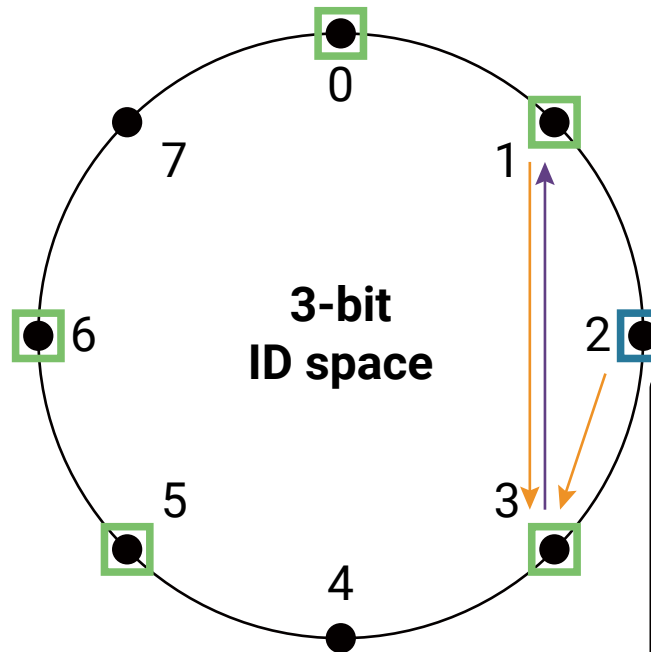


Chord – Node Joining

- Identifiers have $m = 3$ bits
- Key space: $[0, 2^3 - 1]$

● Identifiers/key space

□ Node



→ Points to successor
→ Points to predecessor

Node 2 is joining

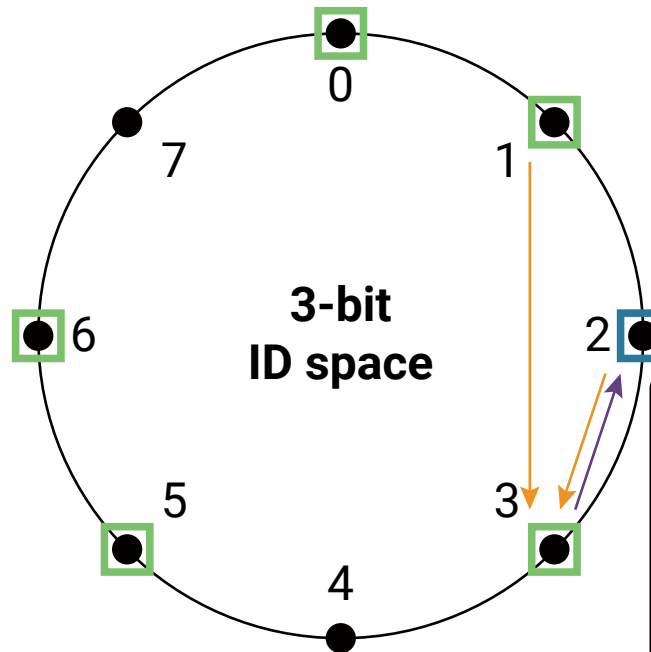
```
STABILIZE() [N.successor=M]
N->M: "What's your predecessor?"
M->N: "x is my predecessor"
if x between (N,M), N.successor=x
N->N.successor: NOTIFY()
NOTIFY()
N->N.successor:
    "You are my successor"
M: upon receiving NOTIFY from N:
    if N between (M.predecessor, M)
        M.predecessor=N
```


Chord – Node Joining

- Identifiers have $m = 3$ bits
- Key space: $[0, 2^3 - 1]$

● Identifiers/key space

□ Node



→ Points to successor
→ Points to predecessor

Node 2 is joining

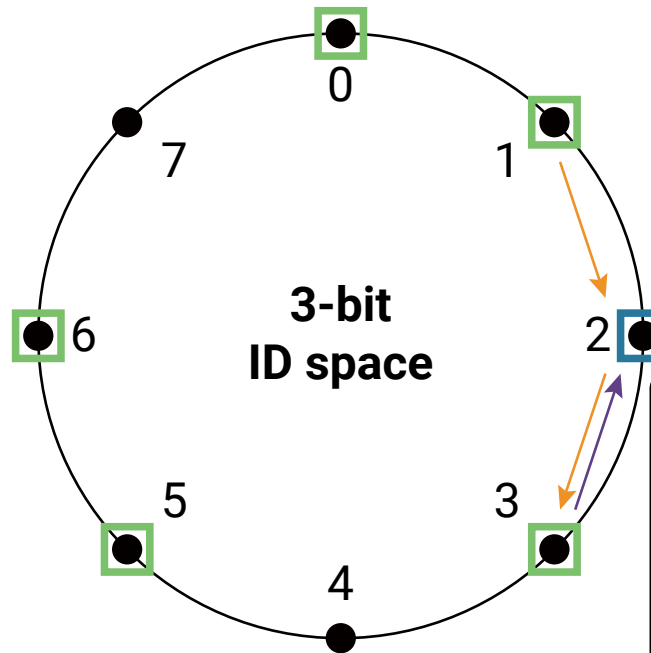
```
STABILIZE() [N.successor=M]
N->M: "What's your predecessor?"
M->N: "x is my predecessor"
if x between (N,M), N.successor=x
N->N.successor: NOTIFY()
NOTIFY()
N->N.successor:
    "You are my successor"
M: upon receiving NOTIFY from N:
    if N between (M.predecessor, M)
        M.predecessor=N
```

Chord – Node Joining

- Identifiers have $m = 3$ bits
- Key space: $[0, 2^3 - 1]$

● Identifiers/key space

□ Node



→ Points to successor
→ Points to predecessor

Node 2 is joining

```

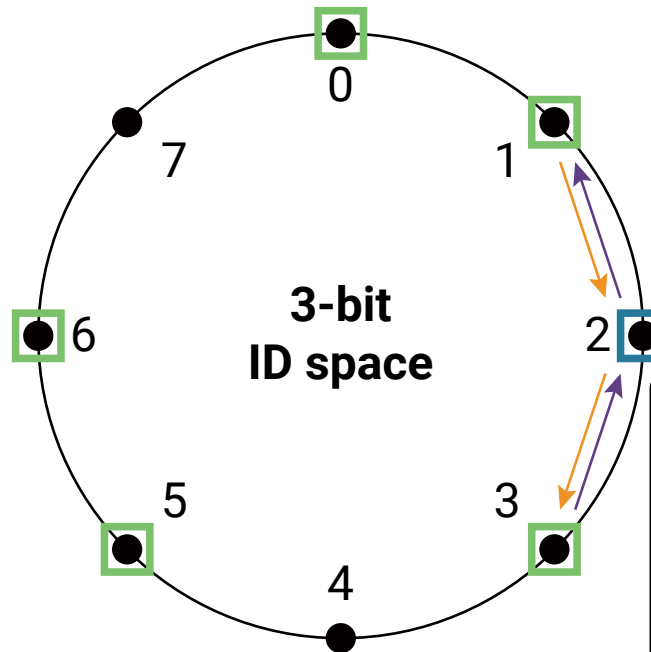
STABILIZE() [N.successor=M]
N->M: "What's your predecessor?"
M->N: "x is my predecessor"
if x between (N,M), N.successor=x
N->N.successor: NOTIFY()
NOTIFY()
N->N.successor:
    "You are my successor"
M: upon receiving NOTIFY from N:
    if N between (M.predecessor, M)
        M.predecessor=N
    
```

Chord – Node Joining

- Identifiers have $m = 3$ bits
- Key space: $[0, 2^3 - 1]$

● Identifiers/key space

□ Node



→ Points to successor
→ Points to predecessor

Node 2 is joining

```
STABILIZE() [N.successor=M]
N->M: "What's your predecessor?"
M->N: "x is my predecessor"
if x between (N,M), N.successor=x
N->N.successor: NOTIFY()
NOTIFY()
N->N.successor:
    "You are my successor"
M: upon receiving NOTIFY from N:
    if N between (M.predecessor, M)
        M.predecessor=N
```



Chord — Failures and Successor List



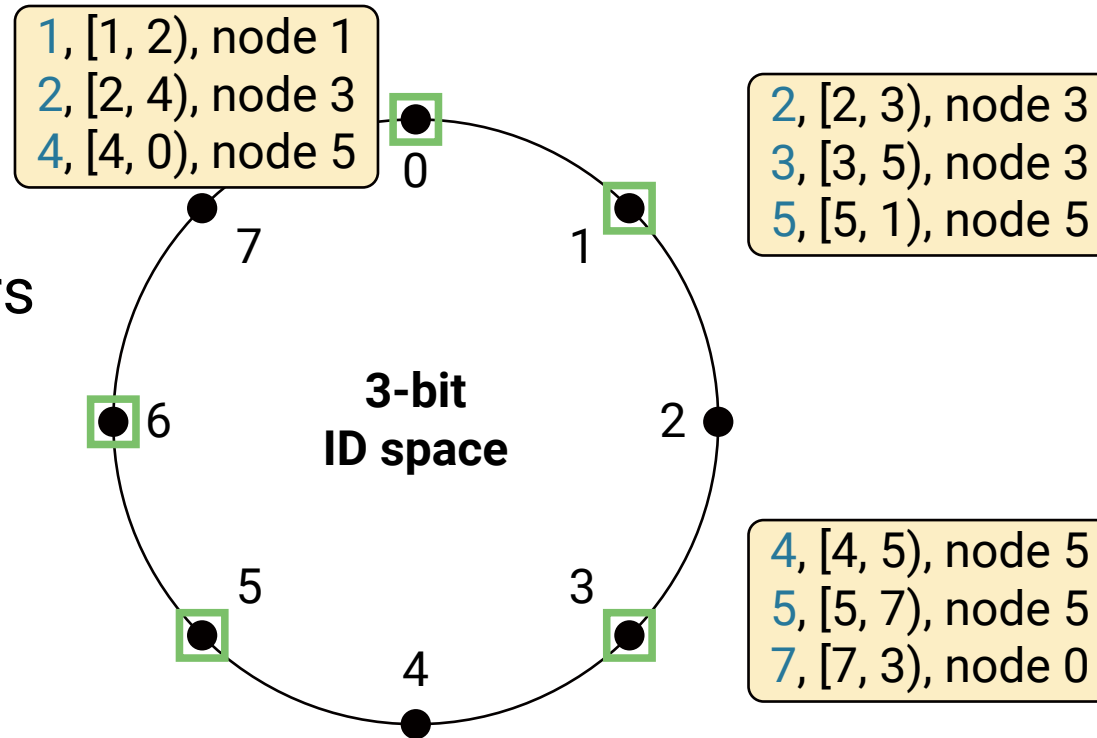
- What if a Node Crashes?
 - If a node's successor fails, the ring is broken, and lookups fail
- **Successor Lists:**
 - Each node maintains a list of its r nearest successors (e.g., $r = 3$)
 - If the immediate successor fails, the node contacts the next one on the list
- **Failure Recovery:**
 - **Predecessor Pointer:** Used to walk backward or verify stabilization. If a predecessor doesn't respond, it is cleared
 - **Periodic Checks:** Nodes ping their successors and predecessors to detect “silent” departures

Chord – Failures and Successor List

● Identifiers/key space

□ Node

- r -nearest successors
 - $r = \log N$



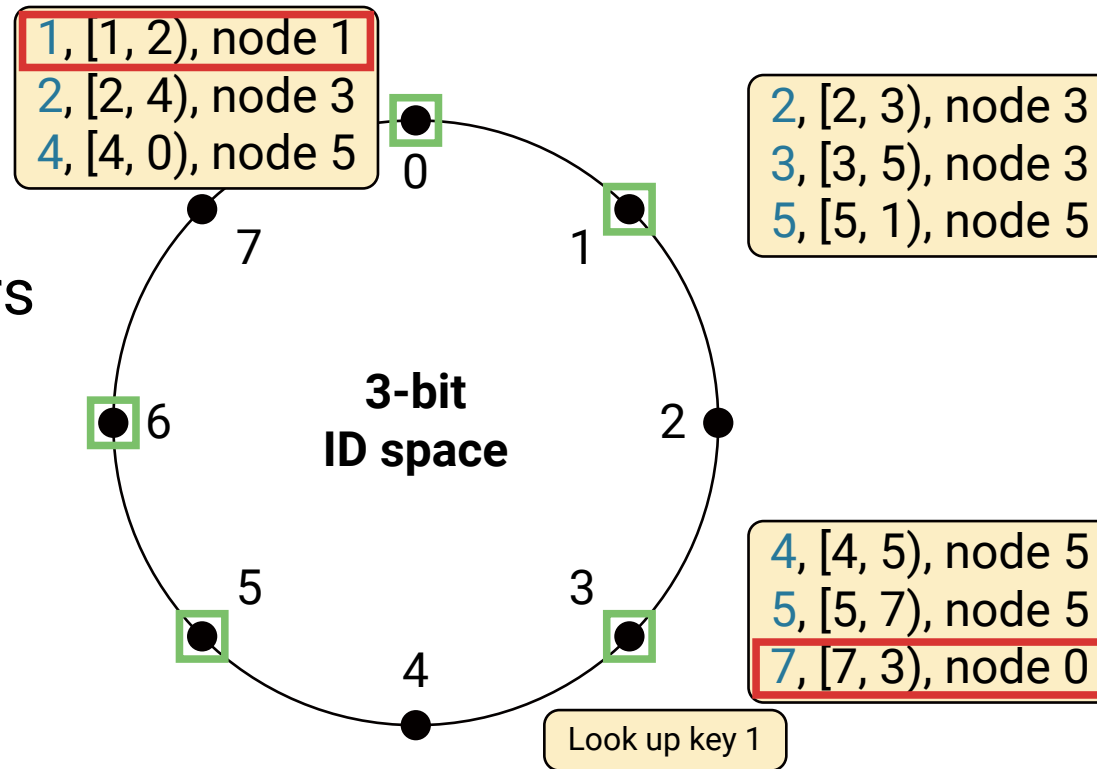
Chord – Failures and Successor List

● Identifiers/key space

□ Node

• r -nearest successors

▸ $r = \log N$

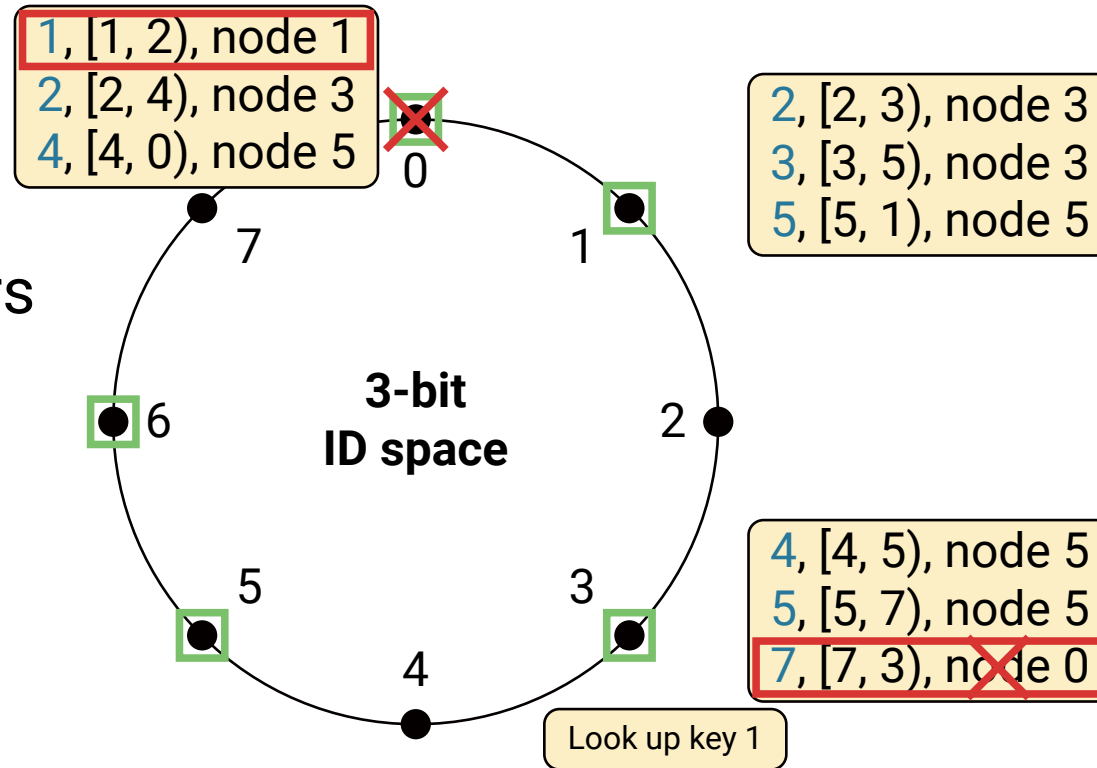


Chord — Failures and Successor List

● Identifiers/key space

□ Node

- r -nearest successors
 - $r = \log N$



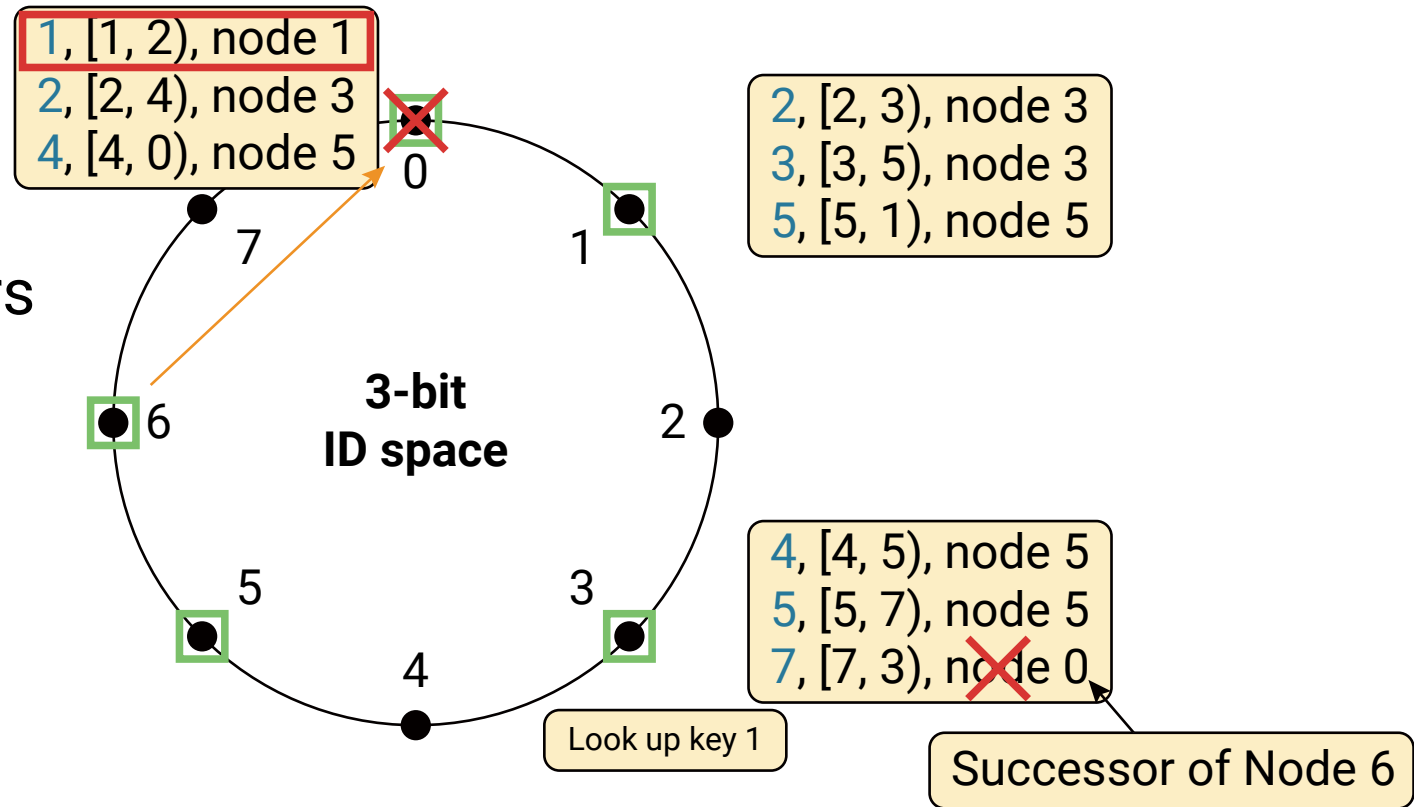
Chord – Failures and Successor List

● Identifiers/key space

□ Node

• r -nearest successors

▸ $r = \log N$

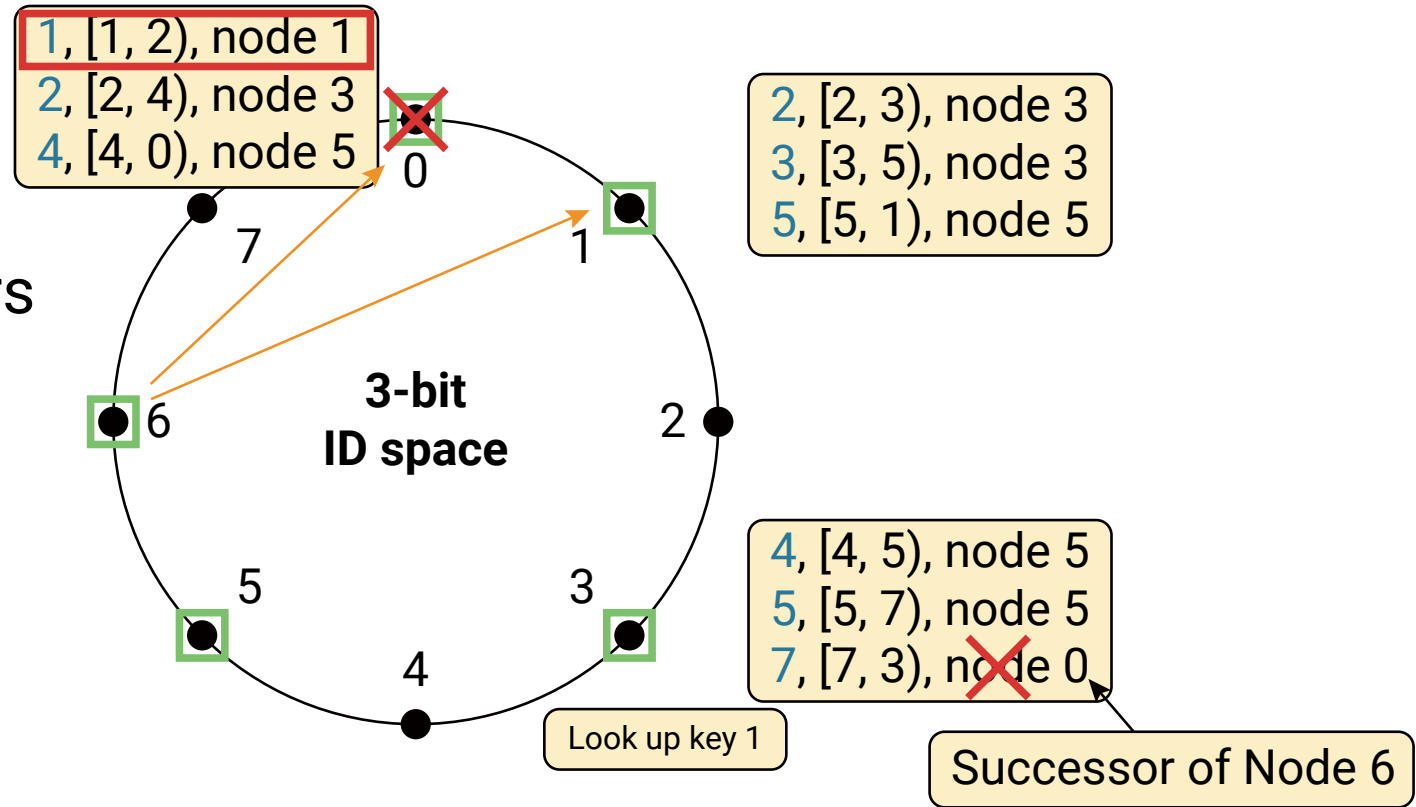


Chord – Failures and Successor List

● Identifiers/key space

□ Node

- r -nearest successors
 - $r = \log N$

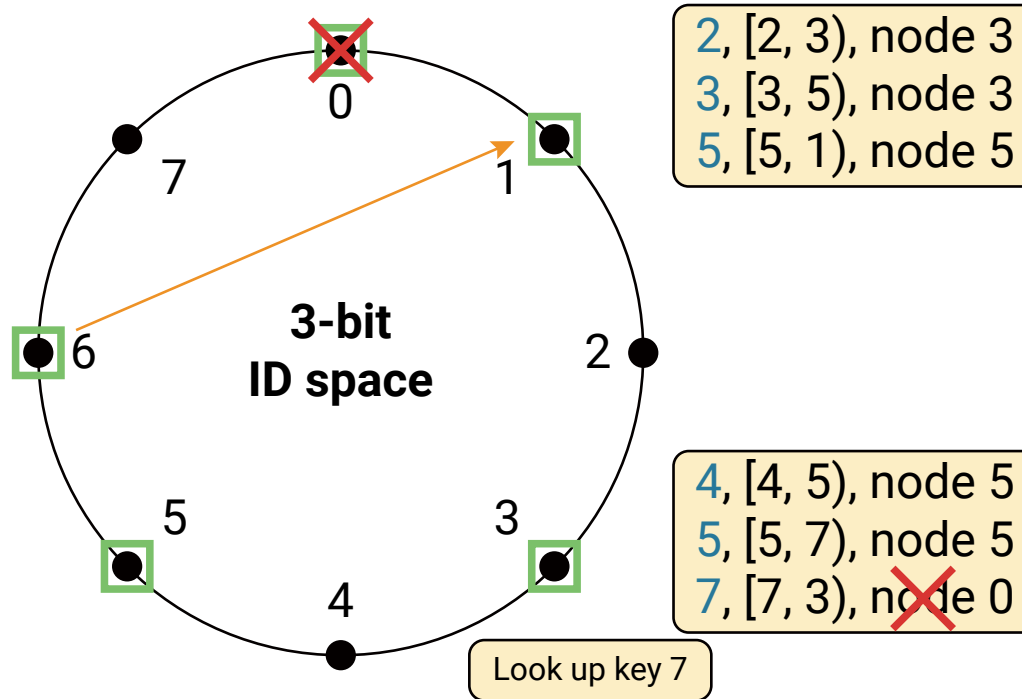


Chord – Failures and Successor List

● Identifiers/key space

□ Node

- r -nearest successors
 - $r = \log N$

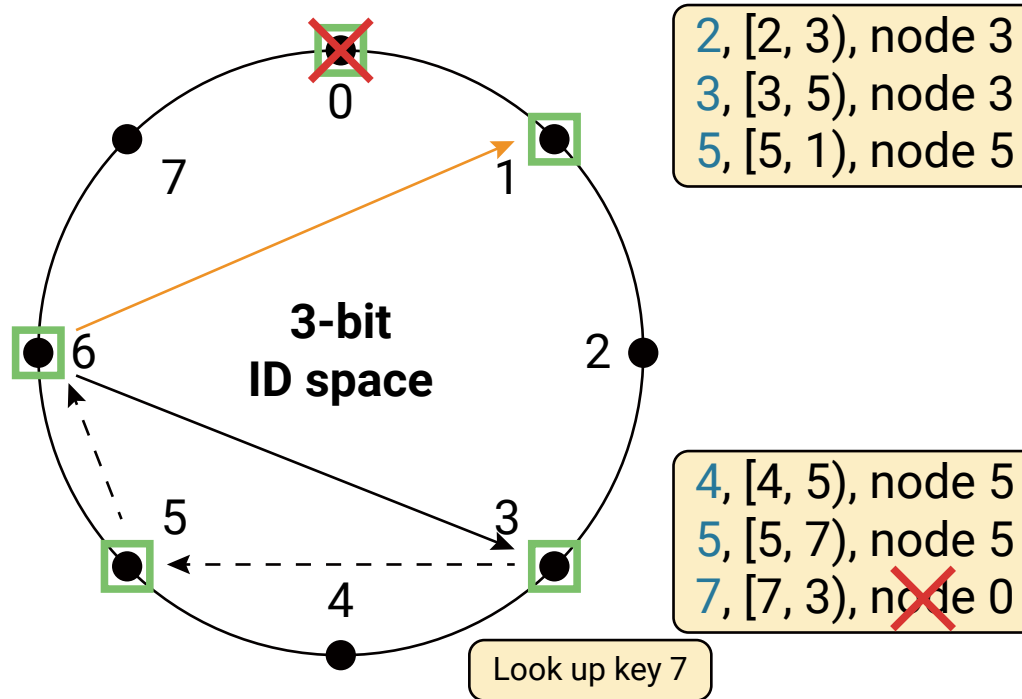


Chord – Failures and Successor List

● Identifiers/key space

□ Node

- r -nearest successors
 - $r = \log N$

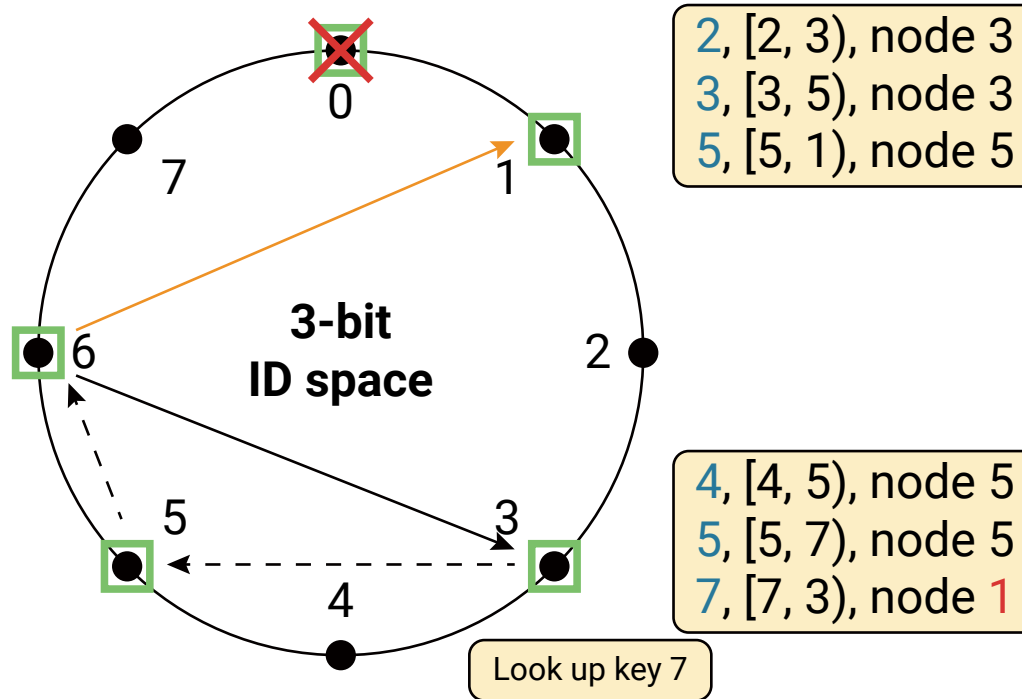


Chord — Failures and Successor List

● Identifiers/key space

□ Node

- r -nearest successors
 - $r = \log N$



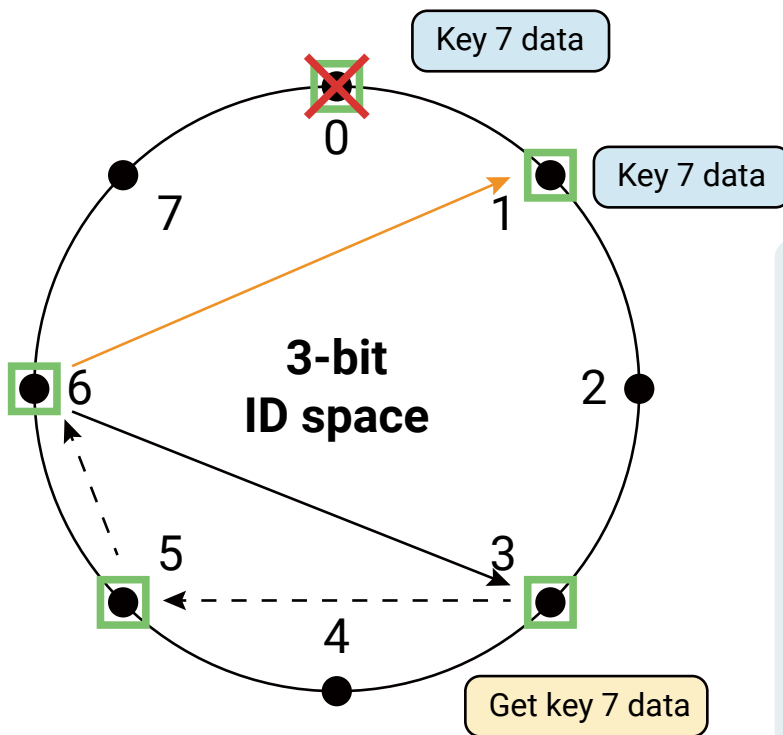
DHash Replicates Data Blocks at r Successors

● Identifiers/key space

□ Node

• r -nearest successors

▸ $r = \log N$



“Adjacent” nodes in the ring may be far away in the network

→ Independent failures

Why Don't All Services Use P2P?

- **High latency and limited bandwidth** between peers (vs. intra/inter-datacenter, client-server model)
 - 1M nodes = 20 hops; 50 ms / hop gives 1 sec lookup latency (assuming no failures / slow connections...)
- User computers are **less reliable** than managed servers
- **Lack of trust** in peers' correct behavior
 - Securing DHT routing hard, unsolved in practice

DHTs in Retrospective

- Seem promising for finding data in large P2P systems
- Decentralization seems good for load, fault tolerance
- But: the **security problems** are difficult
- But: **churn** is a problem, particularly if $\log(n)$ is big
- DHTs have not had the hoped-for impact

What DHTs Got Right

- **Consistent hashing**
 - Elegant way to divide a workload across machines
 - Very useful in clusters: actively used today in Amazon Dynamo and other systems
- **Replication** for high availability, efficient recovery
- **Incremental scalability**
 - Peers join with capacity, CPU, network, etc.
- **Self-management**: minimal configuration



© 2026 James Jianqiao Yu
Harbin Institute of Technology, Shenzhen

All rights reserved.

These lecture materials are intended solely for educational use in the course *Distributed Systems*. Permission is granted to students enrolled in the course to reproduce these materials, in whole or in part, for personal, non-commercial use only (including printing and photocopying). Any further reproduction or distribution, including but not limited to public posting on websites, social media, forums, or file-sharing platforms, distribution to non-enrolled individuals, or any commercial use, is prohibited without the prior written permission of the author.

This slide deck is adapted from Princeton University's *Distributed Systems* course materials, originally created by Michael Freedman, Kyle Jamieson, and Wyatt Lloyd, and licensed under the Creative Commons Attribution-NonCommercial-ShareAlike 3.0 Unported (CC BY-NC-SA 3.0) License. The original authors and source are gratefully acknowledged.